

데이터 분산처리 끝내기

- Part2 Kafka

이 수업에서 다루고자 하는 것

이론

Chapter 1. Kafka란?

- Kafka의 역사, 기본 구조 소개
- 매니지드 or 온프레미스 카프카 어떻게 결정할까?

Chapter 2. Kafka의 구조

- 카프카 브로커의 이해
- 카프카 클러스터와 주키퍼
- 카프카의 토픽과 파티션
- 카프카의 레코드와 세그먼트
- 카프카에서 복제란?
- ISR에 대한 이해

Chapter 3. Kafka 프로듀서

- 카프카 프로듀서의 이해
- 카프카 프로듀서 옵션들
- 카프카 프로듀서 acks

Chapter 4. Kafka 컨슈머

- 카프카 컨슈머의 이해
- 카프카 컨슈머 그룹의 이해
- 카프카 컨슈머 락
- 카프카 컨슈머 옵션들

이 수업에서 다루고자 하는 것

실습

- Local에 Kafka 설치하기
- Local에 Kafka 브로커 실행하기
- Local에 Producer 설정하기
- Local에 Consumer 설정하기
- 실습을 위한 기본 세팅
- API 데이터 카프카로 전송하기

Chapter 1. Kafka란?

Chapter 1.

01. Kafka, 기본 구조 소개

01. Kafka란?



Apache Kafka

- 2011년 LinkedIn에서 개발
- Apache Kafka는 스트림 처리, 실시간 데이터 파이프라인 및 대규모 데이터 통합에 사용되는 오픈 소스 분산 스트리밍 시스템
- 데이터를 생성하는 어플리케이션과 데이터를 소비하는 어플리케이션 간의 중재자 역할

01. Kafka란?

Why Kafka?

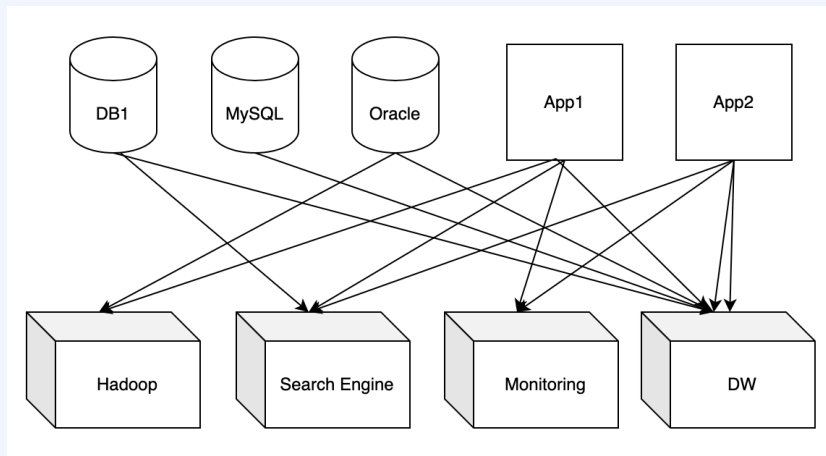
Kafka는 Fortune 100대 기업의 80% 이상이 사용되는 오픈소스

- 높은 처리량
- 높은 확장성
- 낮은 대기시간
- 데이터 보관
- 고가용성



01. Kafka란?

Why Kafka?

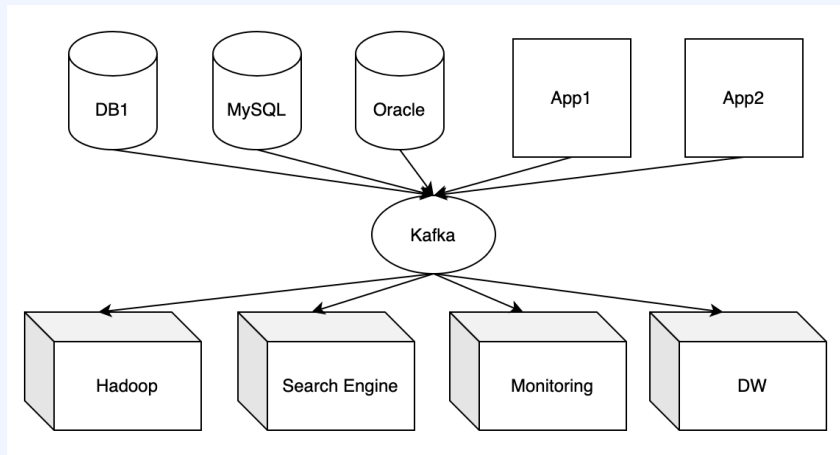


Before Kafka

- 데이터 아키텍처가 파편화 되어 있음
- 시스템이 점점 커질 수록 복잡해지는 문제가 발생함

01. Kafka란?

Why Kafka?

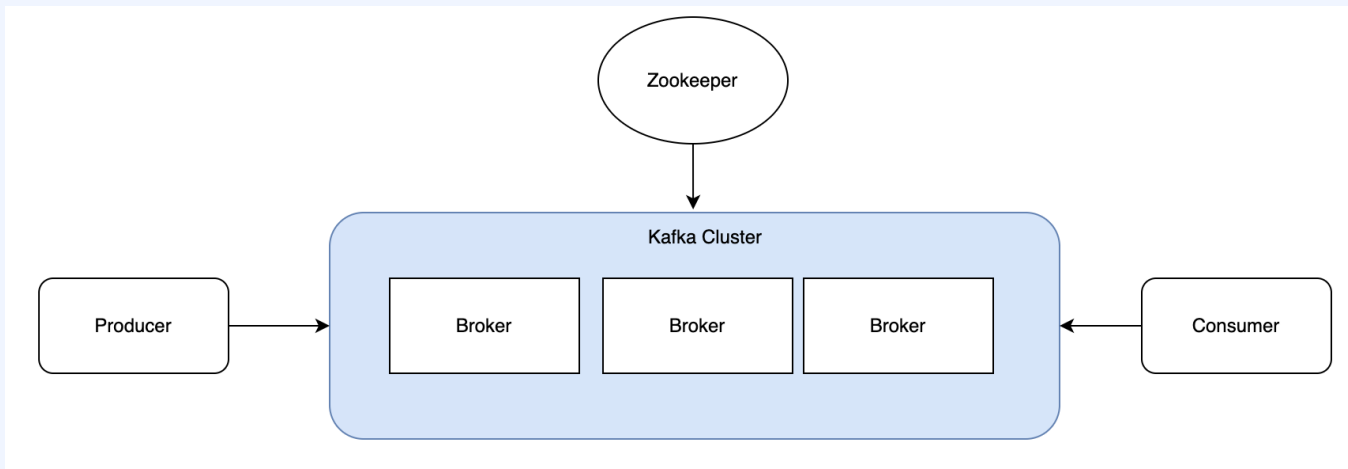


After Kafka

- 데이터 아키텍처가 카프카를 통해 중앙집중화
- 심플한 구조로 시스템이 점점 커져도 데이터는 카프카에서 가져가면 됨
- 소스와 타겟의 커플링을 최소화

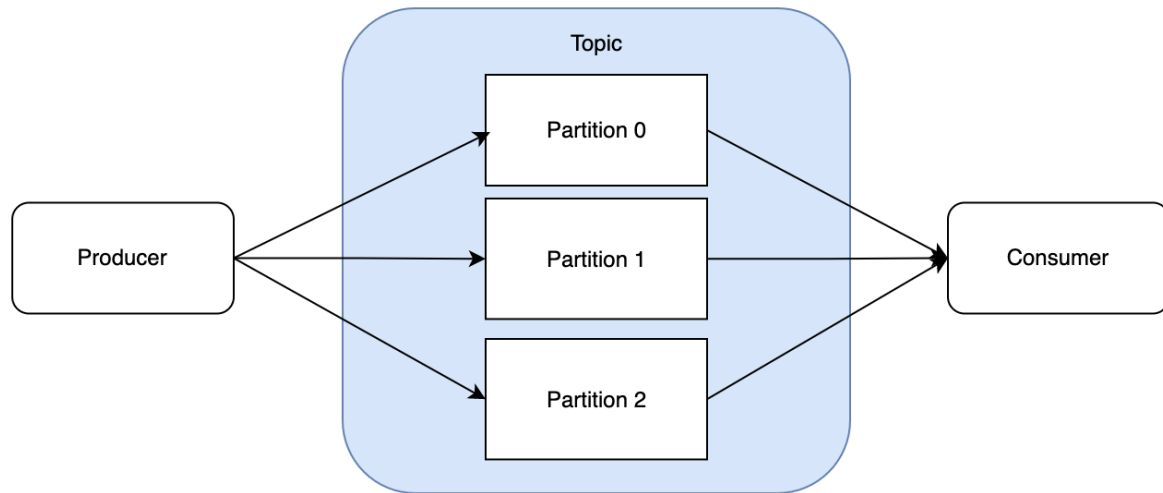
01. Kafka란?

Kafka의 기본 구조



01. Kafka란?

Kafka의 기본 구조



Chapter 1.

02. 매니지드 or 온프레미스 카프카 어떻게 결정할까?

02. 매니지드 or 온프레미스 카프카 어떻게 결정할까?

Managed Service?

매니지드 서비스? 온프레미스?

- 온프레미스 (on-premise) :
 - 서버에 직접 설치해서 운영
 - 기업의 비즈니스 정보를 높은 보안레벨에서 관리가 가능
 - 장기적으로 사용 시 비용 효율 높음
 - 높은 초기 구축 비용
 - 시스템 구축에 별도의 시간 필요
 - 인프라를 관리하기 위한 전담 인력 및 조직 필요

02. 매니지드 or 온프레미스 카프카 어떻게 결정할까?

Managed Service?

매니지드 서비스? 온프레미스?

- 클라우드 서비스 : 필요한 IT리소스를 구입하는 것
- IaaS (Infrastructure-as-a-Service) : 인프라에 필요한 서버나 스토리지를 빌려 사용할 수 있는 서비스 ex) AWS EC2, GCP VM
- PaaS (Platform-as-a-Service) : 개발하고 구축하고 실행하는데 필요한 개발 도구와 환경을 제공하는 서비스, 클라우드에서 완전한 개발 및 배포 환경을 제공 ex) GCP App Engine
- SaaS (Software-as-a-Service) : 소프트웨어를 클라우드에서 구매해서 사용할 수 있음 ex) AWS MSK

02. 매니지드 or 온프레미스 카프카 어떻게 결정할까?

Managed Service?

매니지드 서비스? 온프레미스?

- 클라우드 서비스 :
 - 초기 투자 거의 비용 없음
 - 데이터가 증가할수록 추가적인 비용
 - 빠르게 도입 가능
 - 관리 전문 인력 불필요
 - 클라우드 사의 보안시스템

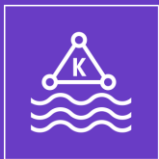
02. 매니지드 or 온프레미스 카프카 어떻게 결정할까?

Managed Kafka?

클라우드 서비스에서 제공되는 카프카 (SaaS)



CONFLUENT



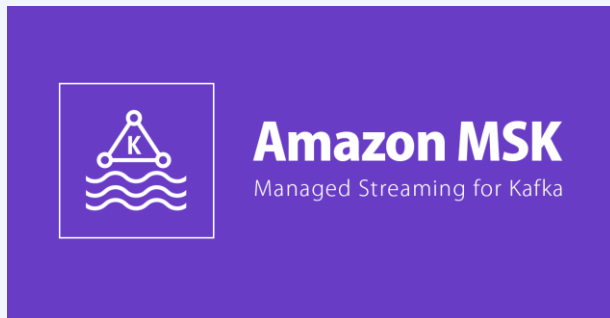
Amazon MSK

Managed Streaming for Kafka

02. 매니지드 or 온프레미스 카프카 어떻게 결정할까?

Managed Kafka의 장점

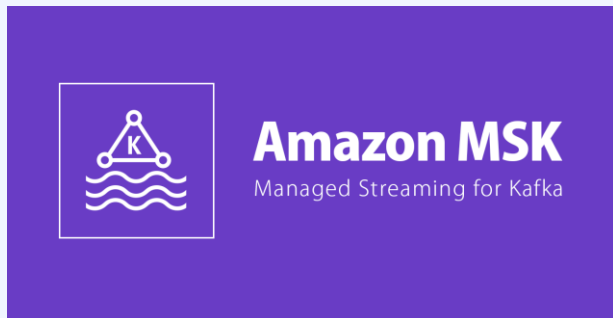
- **인프라 관리**
운영자가 크게 할 게 없음
자동으로 클러스터 복구
스케일 인 아웃 쉬움
- **지표를 보기 쉽다**
내재된 대시보드 제공
지표수집 쉬움
- **보안 - 인증서 자동으로 갱신**
SSL, SSAL, ACL 등 보안설정 제공
접속 보안 설정 기본



02. 매니지드 or 온프레미스 카프카 어떻게 결정할까?

Managed Kafka의 단점

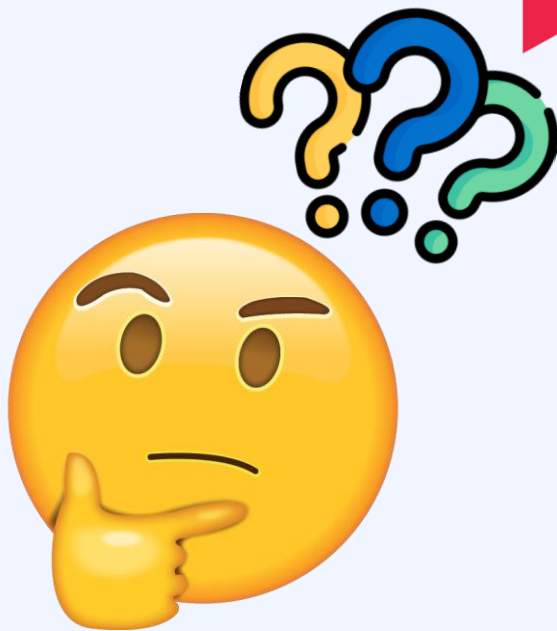
- 비용
AWS나 Confluent에 요금을 지불해야함
직접 서버에 올리는게 저렴
스토리지 비용, 네트워크 비용
- 커스터마이징
온프레미스의 경우 다양한 사용자 설정 가능
상세 설정 불가능, 업체 설정을 따라감
- 클라우드 락인
클라우드 락인, 특정 업체의 서비스나 솔루션에 종속되는 것
상황에 따른 유연한 변경이 불가능



02. 매니지드 or 온프레미스 카프카 어떻게 결정할까?

Managed Kafka, On Premise Kafka

- **빠르게 확장해야 하는 경우**
당장 빠르게 Kafka를 도입해야한다
언제 전문 인력 채용? 양성?
- **전문 인력이 없는 경우**
운영 문제는 누가 관리하지?
에러 핸들링 가능한 인력이 없다면
- **카프카에 대한 노하우**
직접 운영해보고 매니지드로
매니지드는 빠르게 도입이 가능함
카프카 운영 관리, 이슈에 대한 노하우 습득



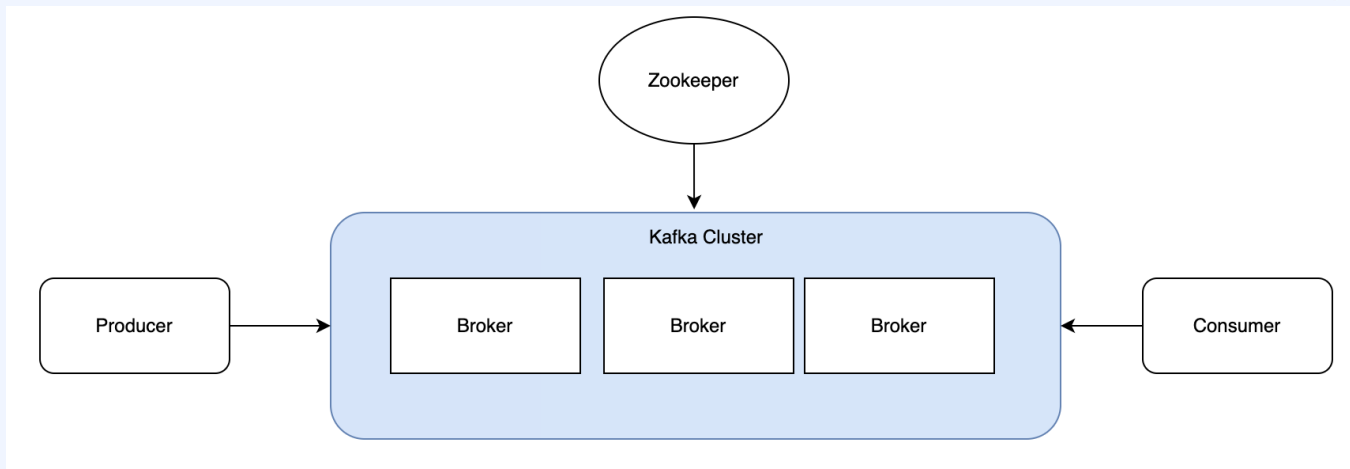
Chapter 2. Kafka의 구조

Chapter 2. Kafka의 구조

01. 카프카 브로커의 이해

01. 카프카 브로커의 이해

카프카 브로커



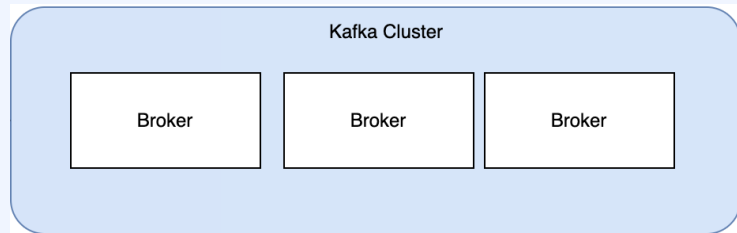
- 카프카 클러스터 내에 존재하는 서버
- 카프카 클라이언트와 데이터를 주고받는 주체
- 데이터를 분산 저장, 복제
- 기본 3대 이상의 브로커 서버를 1개의 클러스터

01. 카프카 브로커의 이해

데이터 저장

- 프로듀서로부터 데이터를 받으면 토픽의 파티션에 데이터를 저장
- 파일 시스템에 저장함 (실습에서 확인)
- 파일 시스템에 저장하므로 File I/O로 인해 속도 이슈 발생가능
- Page Cache를 통해 디스크 입출력 속도를 높임

- 컨슈머가 데이터 요청 시, 파티션에 저장된 데이터를 전달

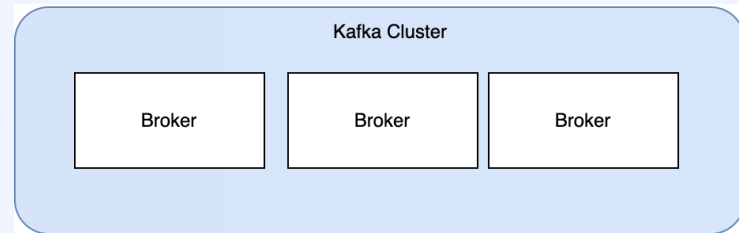


- * 페이지캐시: 파일 입출력 성능 향상을 위해 OS에 만든 메모리 영역
읽은 파일 내용은 이 캐시에 저장하고, 동일 파일을 다룬다면 캐시 메모리에서 직접 읽는다.
이 때문에 힙 메모리 사이즈를 크게 설정하지 않아도 된다.

01. 카프카 브로커의 이해

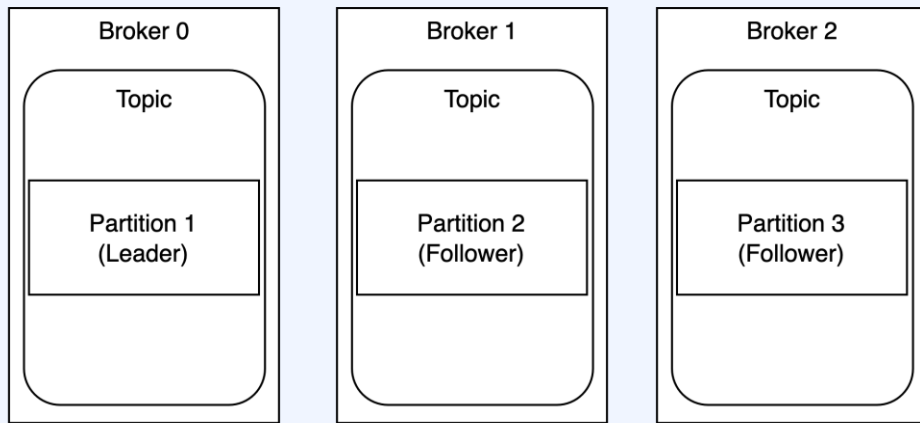
데이터 복제

- 데이터 복제를 통해 고가용성 달성
- 장애 허용 시스템
- 카프카의 데이터 복제 - 파티션 단위
- Replication Factor 설정
- 리더 팔로워 방식



01. 카프카 브로커의 이해

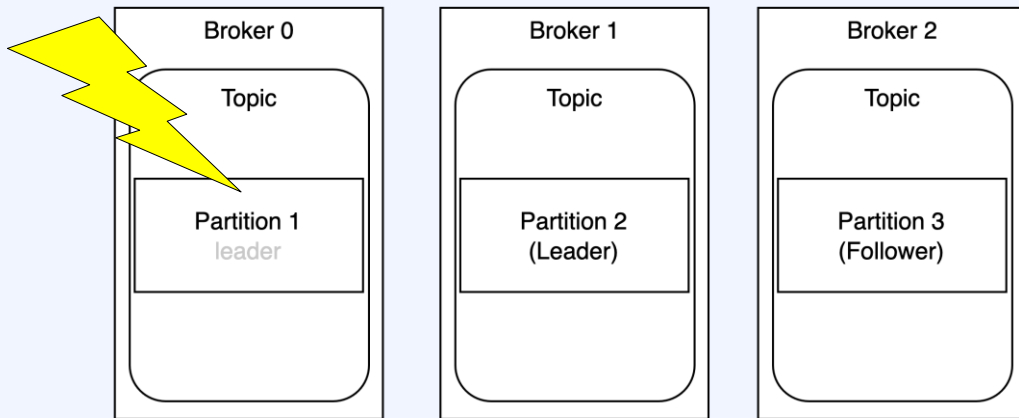
데이터 복제



- 팔로워는 리더 파티션 오프셋 확인, 차이가 나면 리더로부터 가져와 저장
- 2 이상 replication factor 설정을 권장

01. 카프카 브로커의 이해

데이터 복제



- 리더인 0번 브로커에 장애가 난 경우
- FailOver가 일어남 (Part1 참고)
- 팔로우 중 하나가 후보, 리더로 선정됨

01. 카프카 브로커의 이해

컨트롤러

- 브로커의 health체크
- FailOver 과정, Leader 선정
- 컨트롤러 역할을 하는 브로커에 장애가 발생하면 다른 브로커가 이 역할을 맡음

코디네이터

- 컨슈머 그룹 상태 체크
- 파티션과 컨슈머의 매칭, 분배 설정

리밸런싱 : 컨슈머가 컨슈머 그룹에서 빠지게 되면 매칭 되지 않은 파티션을 다른 컨슈머로 할당

01. 카프카 브로커의 이해

데이터 삭제

- 카프카 컨슈머나 프로듀서는 데이터 삭제가 불가능
- 오직 브로커에서만 데이터 삭제가 가능함
- 데이터 삭제는 파일 단위로 이루어짐
- 해당 파일 단위가 로그 세그먼트 (log segment)
- 카프카 브로커에 log.segement.bytes 또는 log.segment.ms 옵션에 값이 설정되면 세그먼트 파일이 닫힘
- 닫힌 세그먼트 파일은 log.retention.bytes 또는 log.retention.ms 옵션에 설정 값이 넘으면 삭제됨

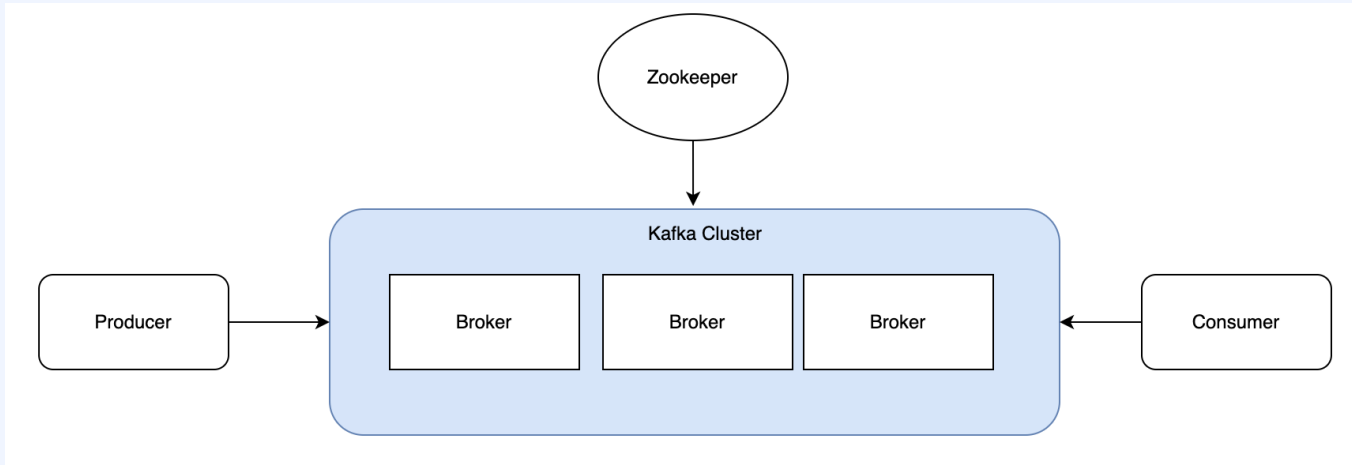
Chapter 2. Kafka의 구조

02. 카프카 클러스터와 주키퍼

02. 카프카 클러스터와 주키퍼

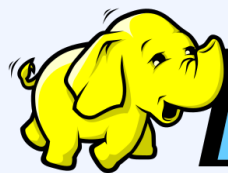
카프카 클러스터

- 브로커들을 하나의 클러스터로 관리
- 브로커, 즉 카프카 서버들의 집합



02. 카프카 클러스터와 주키퍼

Zookeeper?



hadoop

A P A C H E
HBASE



HIVE



Apache Pig



TAJO

02. 카프카 클러스터와 주키퍼

Zookeeper?



<https://ici.net.au/blog/wp-content/uploads/2017/03/Ici-A-Typical-Day-in-the-Life-of-a-Zookeeper-1200x675.jpg>

02. 카프카 클러스터와 주키퍼

Zookeeper?

- 분산 애플리케이션을 위한 코디네이션 시스템
- Kafka에도 사용되고, Nifi, Hbase, Hadoop 등에도 사용됨
- 분산시스템에서 발생가능한 여러 문제들을 핸들링
 - 시스템에서 정보를 어떻게 공유할 것인지
 - 상태를 어떻게 체크할 것인지
 - 분산 서버들간의 잠금을 처리하는 방법에 대한 핸들링
- 부분 실패를 안전하게 처리하기 위한 분산 처리 도구



APACHE
ZooKeeper™

02. 카프카 클러스터와 주키퍼

Zookeeper?

- 부분 실패(Partial Failure)
- 네트워크로 연결된 두 노드 사이에 메시지가 전송된 상황
- 만약 네트워크가 끊겼다면?
- 송신자는 수신자가 메시지를 수신했는지 여부를 모름
- 수신자가 메시지는 받았지만 처리 중 죽었다면?
- 작업의 실패 여부조차 모르게 되는 상황



02. 카프카 클러스터와 주키퍼

Zookeeper?

- 분산 코디네이션 시스템을 제대로 못쓴다면,
- 단일 장애점(Single Point of Failure)가 발생, 시스템의 신뢰성 감소
- 분산 어플리케이션이 조율되지 못한다면 클러스터 자원에 무분별한 write 발생
- **경쟁상태**가 일어나게 될 가능성 높아짐
- 여러 프로세스가 동시에 접근 시도
- 타이밍이나 순서 등이 결과값에 영향을 줄 수 있는 상태
- 데이터의 일관성을 해치는 결과가 나타날 가능성



APACHE
ZooKeeperTM

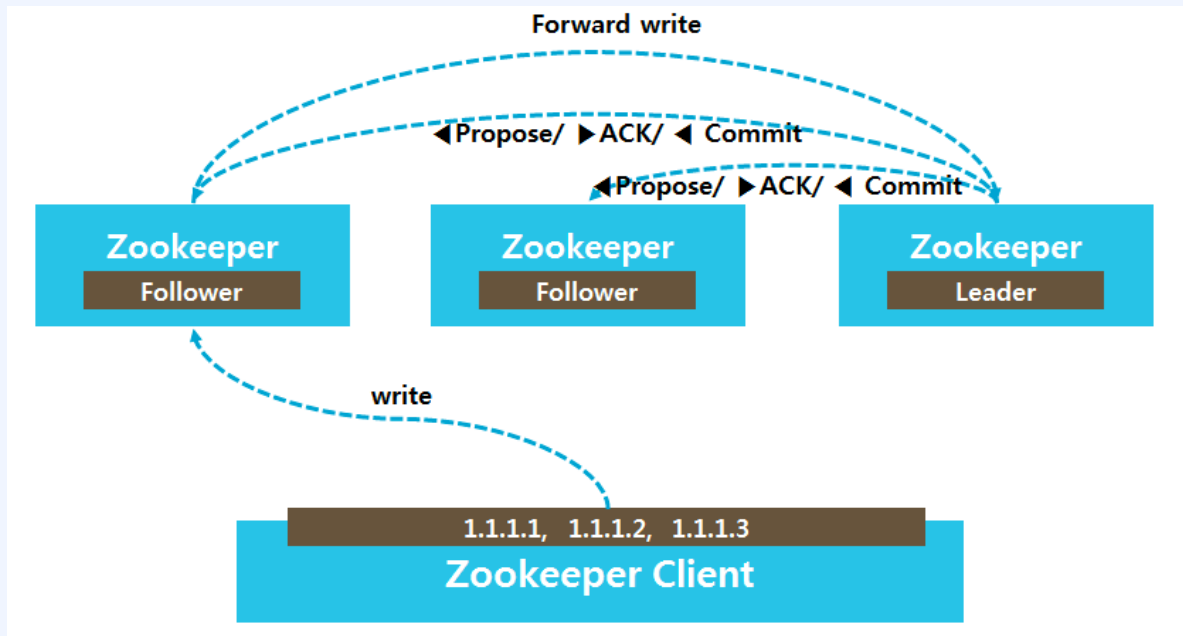
02. 카프카 클러스터와 주키퍼

Zookeeper 사용 목적

- 설정 관리(Configuration management)
클러스터의 설정 정보를 최신으로 유지
- 클러스터 관리(Cluster management)
서버가 추가되거나 제외될 때 정보를 클러스터 내부 서버들에게 공유
- 리더 채택(Leader selection)
여러 후보 노드 중 리더를 선택한다
- 락, 동기화 서비스(Locking and synchronization service)
클러스터에 쓰기 연산이 자주 발생한다면, 경쟁상태에 들어갈 수 있다.
데이터 불일치가 발생할 수 있음.
클러스터 전체에 대해 Lock을 걸어 동기화해 경쟁상태에 들어갈 경우를 사전에 방지함

02. 카프카 클러스터와 주키퍼

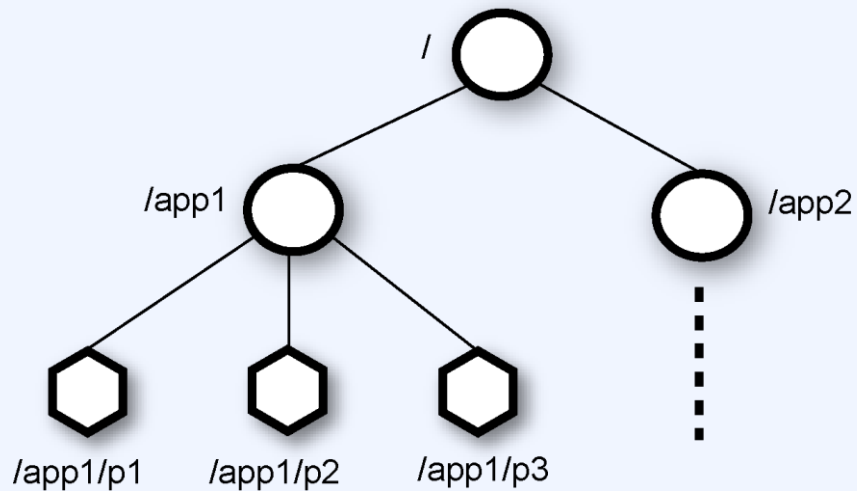
Zookeeper 아키텍처



02. 카프카 클러스터와 주키퍼

Zookeeper 데이터 모델

- Znode



<https://m.blog.naver.com/PostView.naver?isHttpsRedirect=true&blogId=kbh3983&logNo=221082741052>

Chapter 2. Kafka의 구조

03. 카프카의 토픽과 파티션

03. 카프카 토픽과 파티션

카프카 토픽

- 사실상 카프카를 사용하는 이유
- 토픽을 생성하면서 한 파이프라인이 만들어진다
- 카프카에서 데이터를 구분하는 단위
- "구체화된 이벤트 스트림"

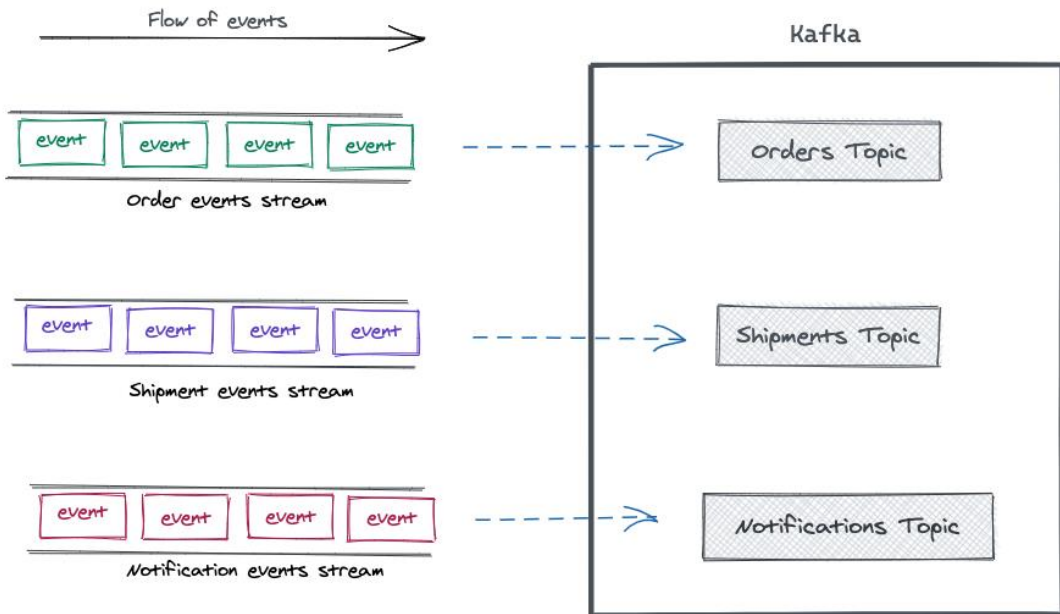
03. 카프카 토픽과 파티션

Events, Streams, and Topics

- Events
어떤 일어난 일, 시스템 사이를 오가는 불변하는 데이터.
레코드 또는 메세지
- Streams
이벤트 스트림이란, 관련된 이벤트들을 뜻함 (순서를 갖고 있음)
- Topic
연관된 이벤트들을 묶어 영속화한 것

03. 카프카 토픽과 파티션

카프카 토픽

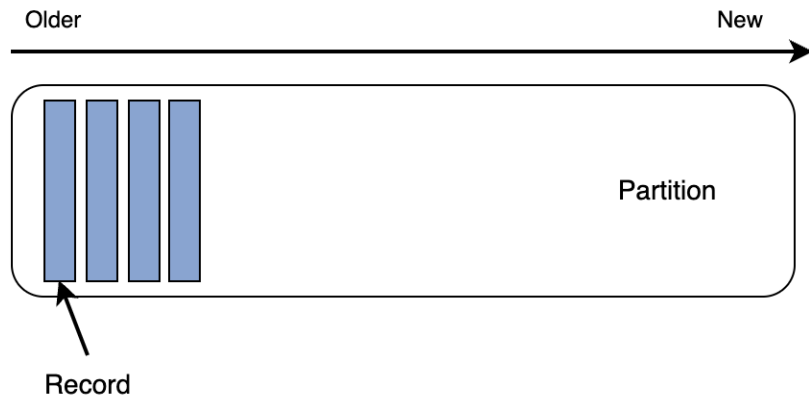


In Kafka, a topic represents a materialized event stream. In other words, a stream at rest.

03. 카프카 토픽과 파티션

카프카 파티션

- 토픽을 이루고 있는 단위
- 토픽에 속한 레코드를 실제 저장소에 저장하는 가장 작은 단위
- Append Only로 저장되는 로그 파일
- 프로듀서가 보낸 데이터들이 파티션에 들어가 저장됨
- 이 데이터는 레코드라고 부름
- 큐와 비슷한 구조 (FIFO)
다만 처리하고도 삭제하지 않음!
- 먼저 들어간 레코드는 컨슈머가 먼저 가져감



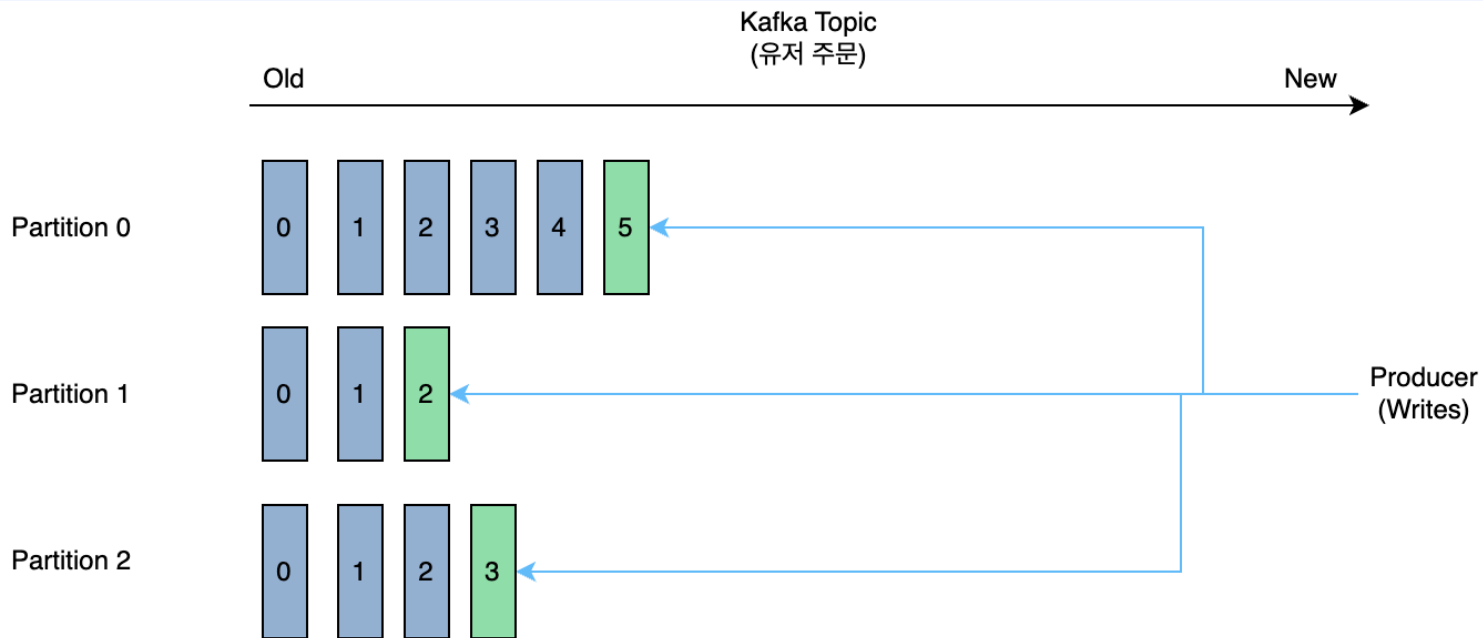
03. 카프카 토픽과 파티션

카프카 파티션

- 파티션의 각 레코드에는 오프셋(Offset)이라는 식별자가 붙음
- Kafka에서 유지 관리하는 증분 및 변경 불가능한 숫자
- 레코드가 파티션에 기록되면 다음 순차 오프셋을 할당하여 로그 끝에 추가
- 파티션 내의 메시지는 순서가 정해져 있지만 토픽 전체의 메시지는 순서가 보장되지 않음

03. 카프카 토픽과 파티션

카프카 파티션



03. 카프카 토픽과 파티션

카프카 파티셔닝의 효과

그림에서 봤듯이 토픽의 파티션을 여러 브로커에 분산시킨다

- 브로커에 파티션을 분산하면 수평적으로 확장하는 효과
싱글 브로커보다 훨씬 뛰어난 성능을 제공
- 다른 컨슈머들이 토픽을 병렬적으로 사용가능
여러 파티션을 설정하면 더 많은 컨슈머를 사용할 수 있게 된다
- 매우 높은 메시지 처리량
각 컨슈머의 인스턴스는 한 파티션에서 메시지를 제공받게됨
각 레코드는 메세지 처리 담당자가 존재하게 된다

파티션은 **Kafka**가 복제하는 방식

- 여러 브로커에 동일한 파티션의 복제본을 2개이상 갖고 있음
- 브로커가 다운되면 다운된 브로커가 갖고있는 파티션의 복제본을 사용함

03. 카프카 토픽과 파티션

카프카 파티셔닝 방법

- 프로듀서는 레코드를 특정 파티션으로 보내야한다
- 방법은 세 가지.
 1. 파티션 키 사용
 2. 카프카가 파티션을 결정
 3. 사용자 지정 파티셔너 설정

03. 카프카 토픽과 파티션

카프카 파티셔닝 방법

파티션 키 사용

- 파티션키를 사용할 데이터를 설정 (ConsumerId, _id)
- Hashing 함수를 통해 전달됨
- 동일 키로 생성한 모든 레코드는 동일 파티션에 저장됨
정확한 순서가 보장될 수 있음
- 키가 잘 분산되지 않을 가능성이 있음
- 한 고객 ID가 전체 트래픽의 70%이상을 발생시킨다면? -> 브로커가 다운될 수 있음
- 분산이 잘 되는지를 잘 확인해야함!

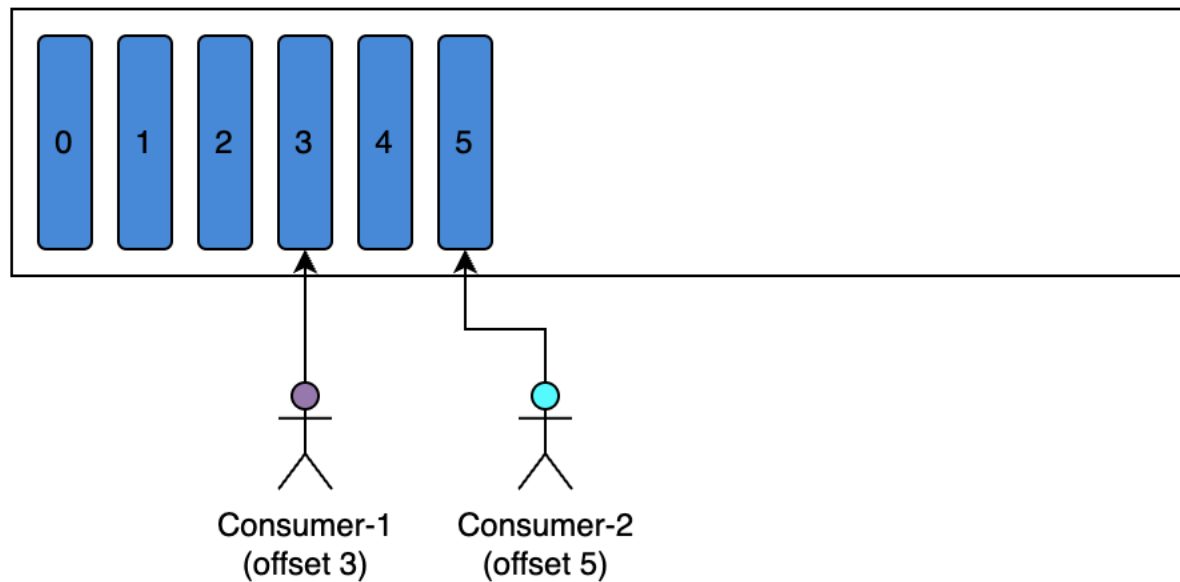
03. 카프카 토픽과 파티션

카프카 파티션에서 레코드 읽기

- 컨슈머가 브로커 파티션에 연결하고 순서대로 메시지를 읽음
 - 오프셋이 컨슈머의 커서로 작동함
 - 컨슈머는 오프셋을 트래킹하면서 사용한 메시지를 추적함
 - 컨슈머는 메시지를 읽고 다음 오프셋으로 이동
 - 각 파티션에 마지막으로 소비된 메시지의 오프셋을 기억함
 - 어떤 시점에 파티션에 들어가더라도 오프셋 위치에서 작업을 재개할 수 있음
 - 하나 이상의 컨슈머가 있더라도 문제 없음
 - 각 컨슈머는 각 파티션에 대해 어디까지 읽었는지 자신만의 기록을 가지게 된다
- 이를 통해 충돌이 나지 않게 됨

03. 카프카 토픽과 파티션

카프카 파티션에서 레코드 읽기



03. 카프카 토픽과 파티션

카프카에 파티션이 많으면 좋을까?

- 파티션 단위의 분산처리
- 파티션이 많다면 동시 처리량이 늘어난다 (높은 throughput)
- 하지만 파티션이 많아진 상태에서 노드가 다운된다면?
토픽 안의 파티션 단위로 분산 처리를 하기 때문에 복구가 복잡하다 => 서비스 리커버리 시간 증가
- 카프카는 모든 디렉토리의 파일에 대한 파일 핸들을 연다
파티션이 많아질수록 리소스가 낭비됨

03. 카프카 토픽과 파티션

적절한 파티션 수는?

- 원하는 목표 처리량의 기준이 필요

ex) 프로듀서 3개에 초당 10개의 메시지를 보낸다면, 토픽에서는 최소한 초당 30개의 메시지를 받아줘야함
1개 파티션이 1초에 10개를 받아준다면, 파티션을 최소 3개로 늘려 이를 처리할 수 있도록 수정

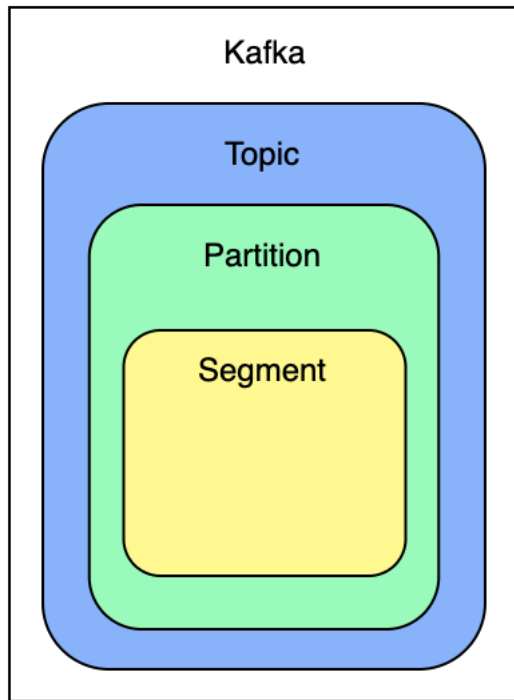
- 파티션을 늘린다음엔 줄일 수 없다
- 따라서 적은 수의 파티션으로 운영을 하면서 늘려주는 것이 이상적

Chapter 2. Kafka의 구조

04. 카프카의 레코드와 세그먼트

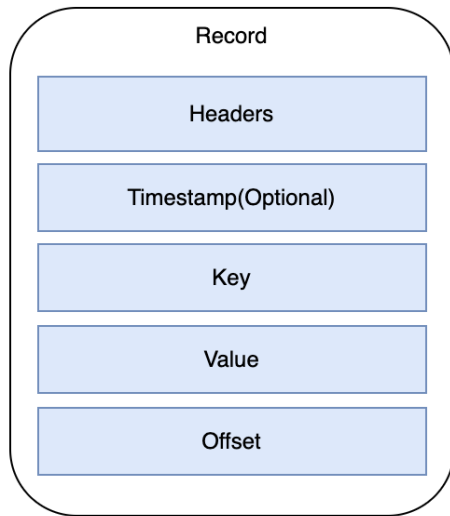
04. 카프카의 레코드와 세그먼트

세그먼트?



04. 카프카의 레코드와 세그먼트

세그먼트 - 레코드의 구성

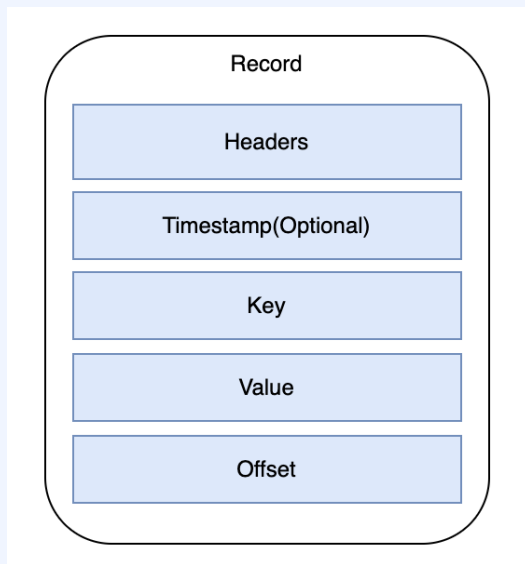


옵션에 따라 생성됨

프로듀서에서 브로커로 데이터가
들어갈 때 offset 지정됨

04. 카프카의 레코드와 세그먼트

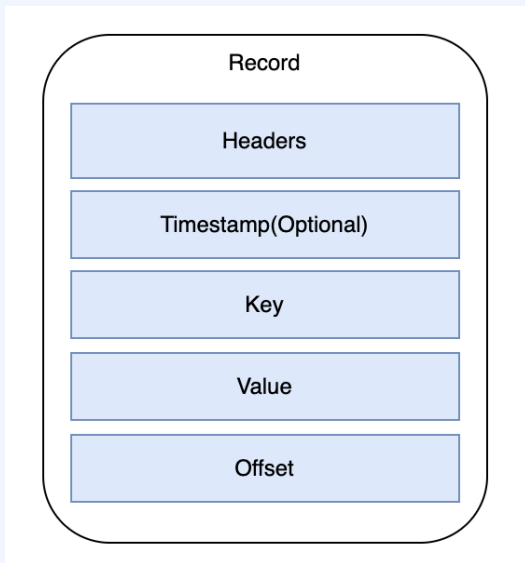
세그먼트 - 레코드의 구성 Header



- Key/Value 데이터를 추가할 수 있다
- 처리에 참고할 정보를 담을 수 있다.

04. 카프카의 레코드와 세그먼트

세그먼트 - 레코드의 구성 Timestamp

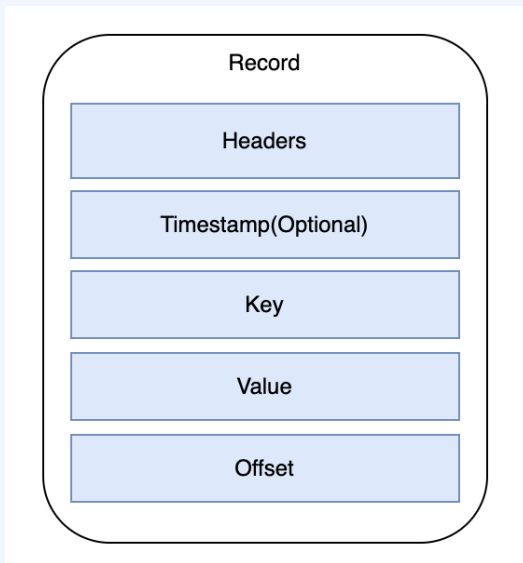


- 시간을 저장함, Unix timestamp가 포함됨
- 기본값은 ProducerRecord 생성시간(Create Time)
- 브로커 적재시간으로 설정가능 (Log Append Time)
- 토픽 단위 설정

04. 카프카의 레코드와 세그먼트

세그먼트 - 레코드의 구성

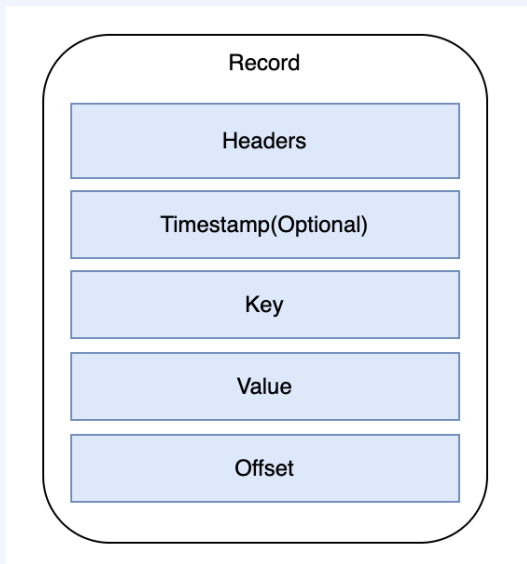
Key



- Key는 분류하기 위한 목적
- 메세지 키는 파티셔너에 의해 토픽의 파티션 번호로 지정됨 (어디로 갈지 지정)
- 키가 없으면 null값, Round Robin방식이 사용됨

04. 카프카의 레코드와 세그먼트

세그먼트 - 레코드의 구성 Value

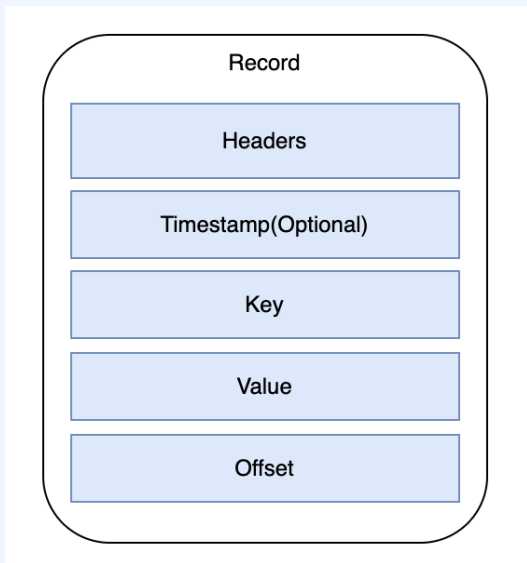


- 실제 값이 들어가는 곳
- Float Byte Array, String 등 지정가능
- 어떤 포맷으로 직렬화 되었는지 컨슈머는 알지 못함, 역직렬화 포맷을 미리 알고 있어야 함
- 보통 String으로 직렬화 역직렬화 또는 Json

04. 카프카의 레코드와 세그먼트

세그먼트 - 레코드의 구성

Offset



- 프로듀서가 생성한 레코드에는 존재하지 않음
- 브로커에 적재되면서 Offset이 지정됨
- 오프셋 기반으로 컨슈머가 처리

04. 카프카의 레코드와 세그먼트

세그먼트?

- 세그먼트(로그 세그먼트)는 파티션에서 확장된 개념
- 프로듀서가 전송한 메시지가 브로커의 로컬 디스크에 저장되는데 이때 저장되는 파일
- 메시지의 키, 오프셋, 크기에 대한 정보도 함께 저장됨
- 밑에 스크린 샷처럼 데이터가 저장됨

```
.  
├── 00000000000000000000000000000000.index  
├── 00000000000000000000000000000000.log  
└── 00000000000000000000000000000000.timeindex
```

04. 카프카의 레코드와 세그먼트

세그먼트?

```
.  
├── 00000000000000000000000000000000.index  
├── 00000000000000000000000000000000.log  
└── 00000000000000000000000000000000.timeindex
```

- Log에는 메시지(레코드)와 메타데이터가 저장됨
- Index에는 오프셋을 인덱싱한 데이터
- Timeindex 메시지에 포함된 timestamp값 기준으로 인덱싱한 정보
메세지키, 메세지 값이 언제의 timestamp로 기록되었는지

04. 카프카의 레코드와 세그먼트

세그먼트?

- 저장될 때는 0000000000.log, 0000000010.log, 0000000020.log ...
- 0, 10, 20 오프셋 번호 (0~9, 10~19, 20~29 ...)
- 가장 앞에 있는 오프셋을 기준으로 파일이름이 정해짐
- 바이트 단위, 시간 단위 기준으로 다음 로그 파일이 저장됨
- log.segment.bytes : 바이트 단위 최대 세그먼트 크기 지정 (1GB)
- log.roll.ms(hours) : 다음 파일로 넘어가는 주기 설정 (7일)

04. 카프카의 레코드와 세그먼트

세그먼트?

- 0000000000.log, 0000000010.log, 0000000020.log 이 있는 상태
- 프로듀서가 데이터를 보내면?
- 20.log에 데이터가 들어가게됨. 20.log 파일은 active한 상태
- 메시지를 쓰기 위해 사용 중인 세그먼트를 **액티브 세그먼트**라고 부른다
- 액티브 세그먼트는 기간이 다 되어도 삭제되지 않는다

04. 카프카의 레코드와 세그먼트

세그먼트와 삭제

- 특정 레코드에 대한 삭제는 불가능
- 삭제 단위는 무조건 세그먼트!
- retention.ms : 레코드 보유 기간. (7일, 저장용량 주의)
 - * disk pressure 오는 경우 이 옵션을 수정하기도 함
- retention.bytes : 지정한 크기에 도달하면 세그먼트를 삭제함 (기본값은 지정되지 않음)
액티브 세그먼트는 삭제되지 않음
- log.retention.check.interval.ms : 브로커에서 세그먼트가 삭제될만한지 체크하는 주기

04. 카프카의 레코드와 세그먼트

세그먼트의 수정

- 이미 적재된 레코드에 대해서는 수정이 불가능함
- 데이터 검증을 미리 하고 적재하는 것이 중요
- 데이터 적재하는 프로듀서측에서 검증하고 데이터를 사용하는 컨슈머에서도 검증필요

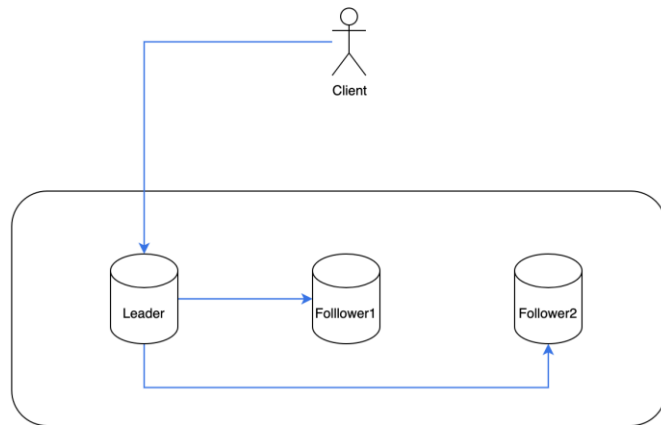
Chapter 2. Kafka의 구조

05. 카프카의 복제란?

05. 카프카의 복제란?

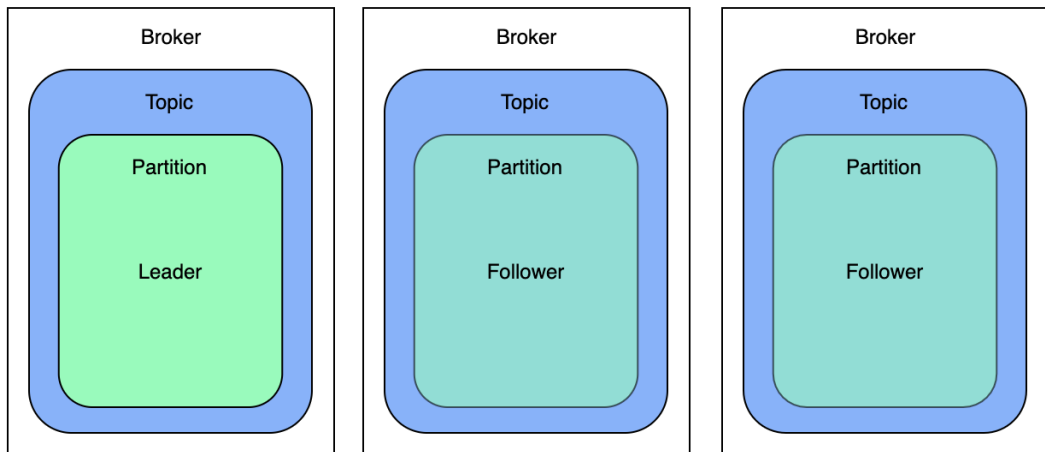
카프카의 복제

- Part1에서 설명한 내용을 잠시 인용하자면...
- 복제를 하는 이유는?고가용성을 위한 것
한 서버가 고장 나면 다른 서버에서 처리할 수 있도록
- 카프카에서의 복제 -> 브로커 장애에 대비
데이터 유실이 발생하지 않도록
데이터를 분산해서 저장한다



05. 카프카의 복제란?

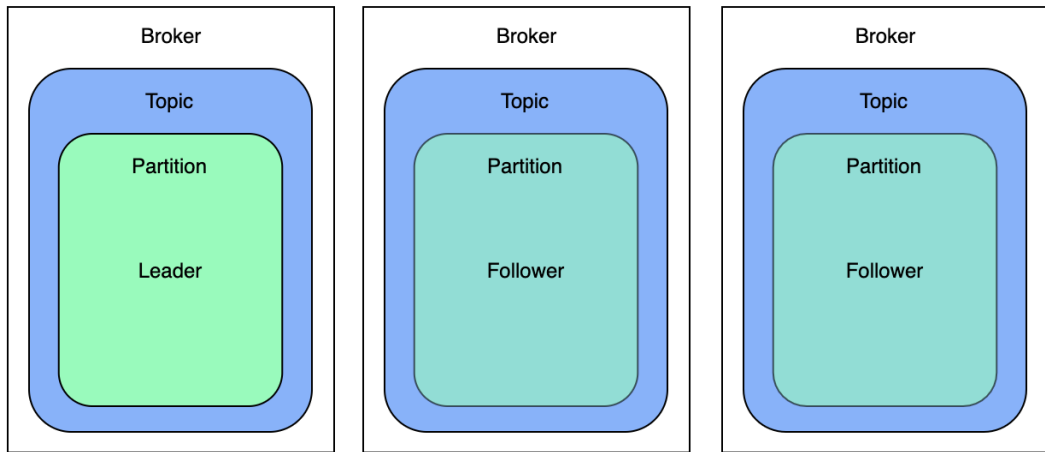
카프카의 복제



- Replication Factor를 통해 복제 개수를 정할 수 있음
- 최소값은 1, 보통 3 이상으로 설정해서 운영
- 최대값은 브로커 개수만큼 할 수 있지만, 보통 2-3

05. 카프카의 복제란?

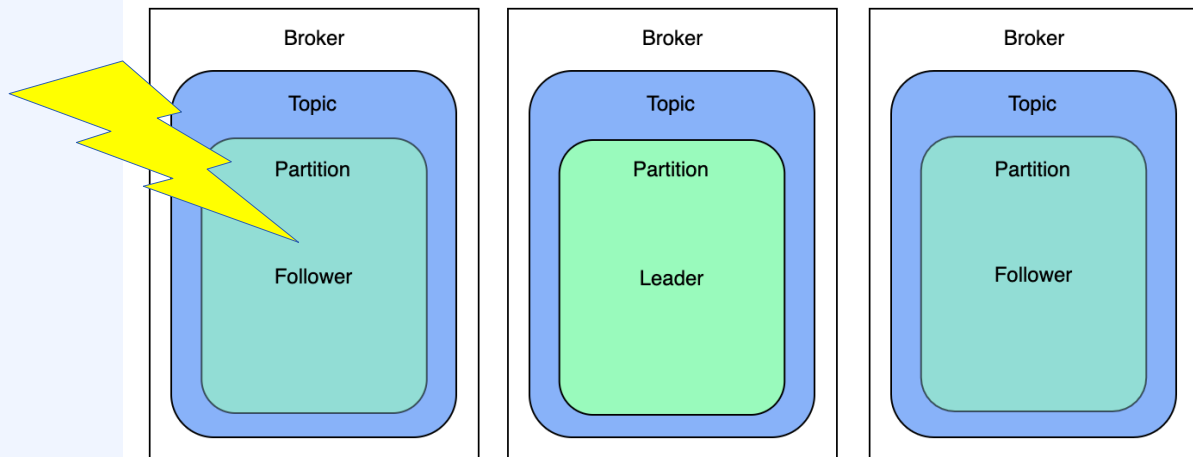
카프카의 복제



- Leader Follower 방식
- 리더는 컨슈머, 프로듀서와 직접 통신한다
- 리더 파티션의 오프셋을 보고 차이나는 부분을 복제해서 “따라잡는다.”

05. 카프카의 복제란?

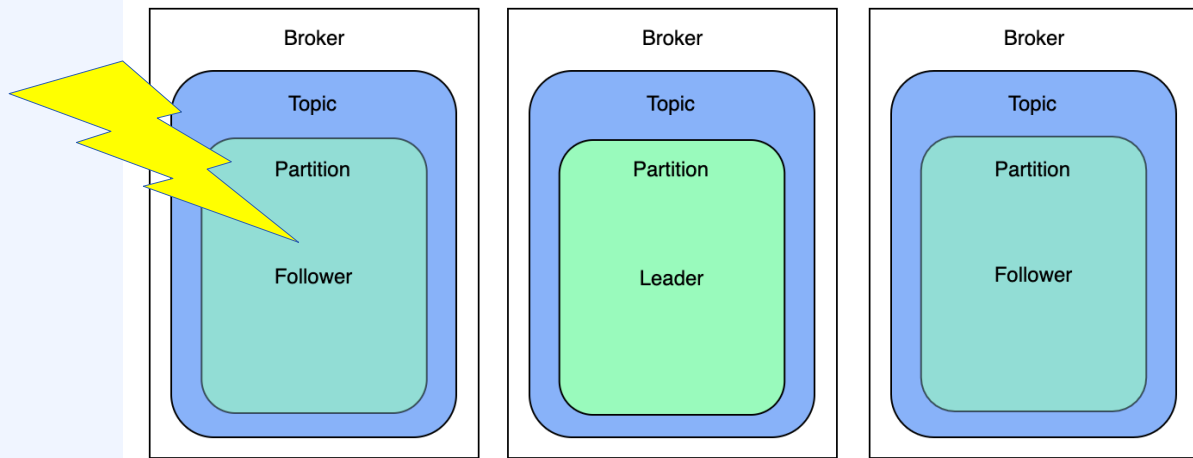
카프카의 복제



- 브로커에 문제가 발생하면 팔로워 파티션 중 하나가 리더가 된다 (승격)
- 리더를 통해 계속 프로듀서와 컨슈머와 통신
- 리더가 선정되는 팔로워는 가장 최신의 데이터를 갖고 있는 팔로워 파티션

05. 카프카의 복제란?

카프카의 복제



- Replication Factor 설정 방법
- 데이터 처리 속도가 중요한 경우 1-2
- 유실에 예민한 경우 3

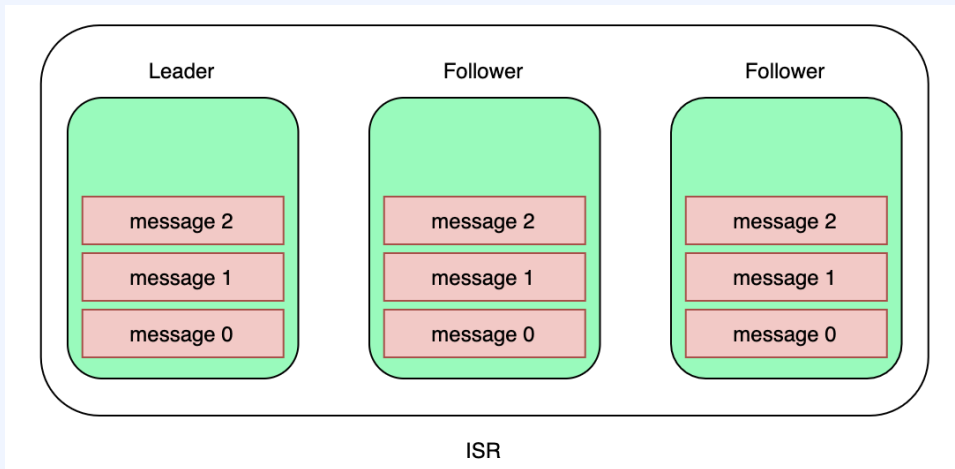
Chapter 2. Kafka의 구조

06. ISR에 대한 이해

06. ISR에 대한 이해

ISR (In Sync Replica)

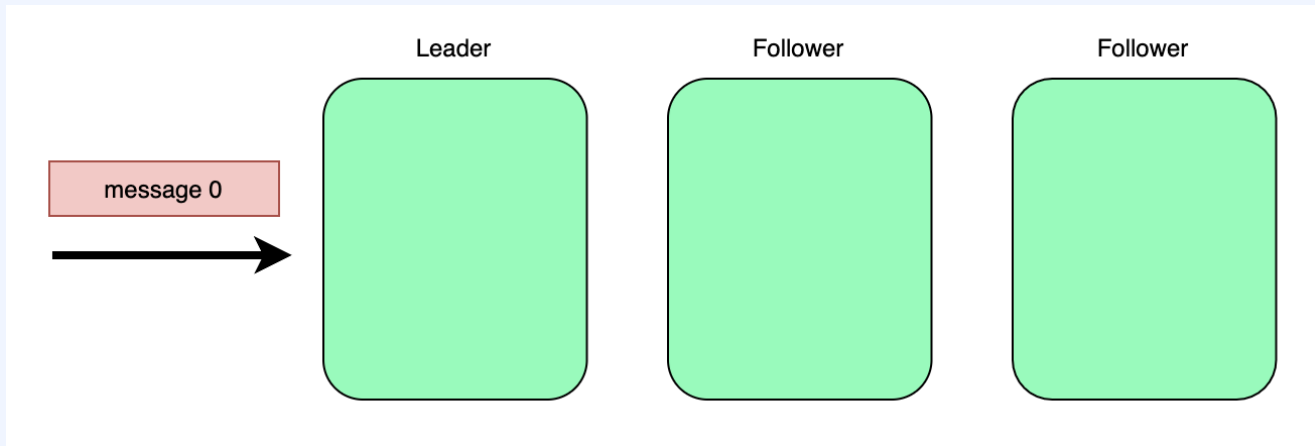
- 복제에서 살펴봤듯이 리더 파티션과 팔로워 파티션이 존재
- 리더 파티션과 팔로워 파티션의 오프셋이 일치하는 상태를 **ISR**이라고 한다
- 현재 리플리케이션이 되고 있는 리플리케이션 그룹(replication group)을 의미



06. ISR에 대한 이해

ISR (In Sync Replica)

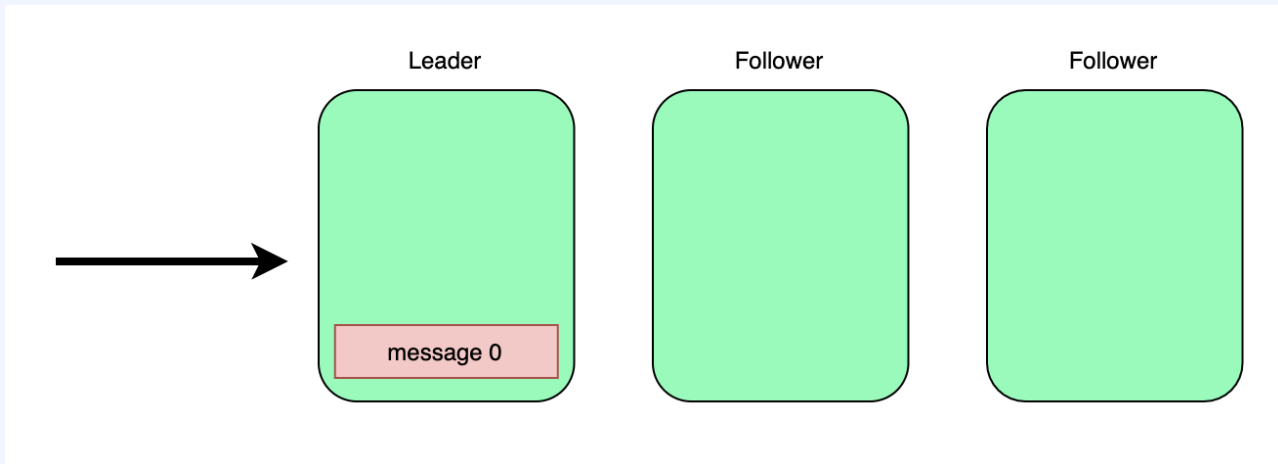
- 메시지가 들어오는 상황을 자세히 살펴보자



06. ISR에 대한 이해

ISR (In Sync Replica)

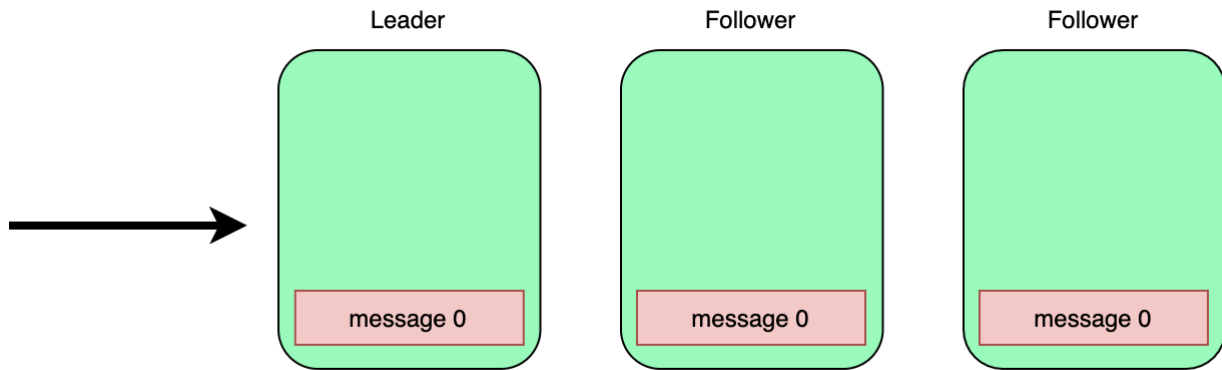
- 메시지가 들어오는 상황을 자세히 살펴보자



06. ISR에 대한 이해

ISR (In Sync Replica)

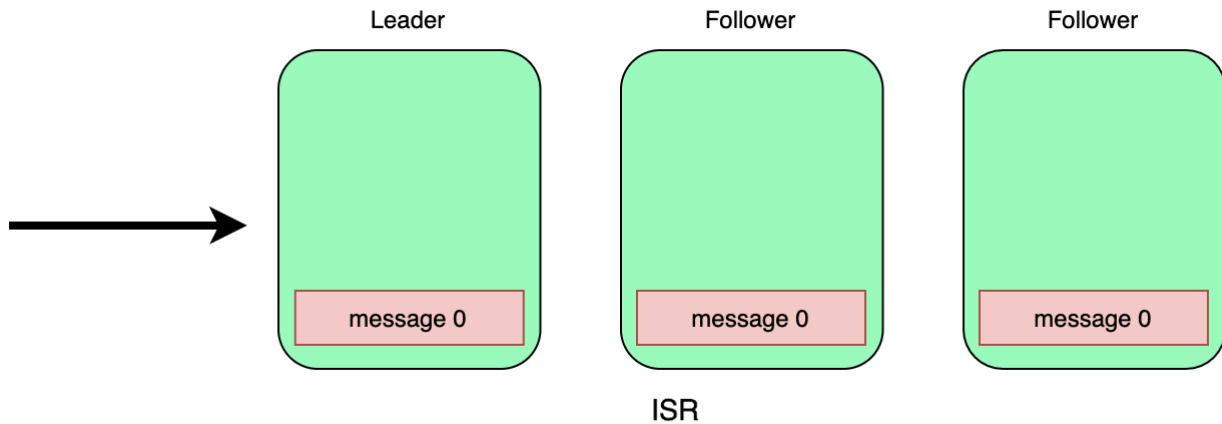
- 메시지가 들어오는 상황을 자세히 살펴보자



06. ISR에 대한 이해

ISR (In Sync Replica)

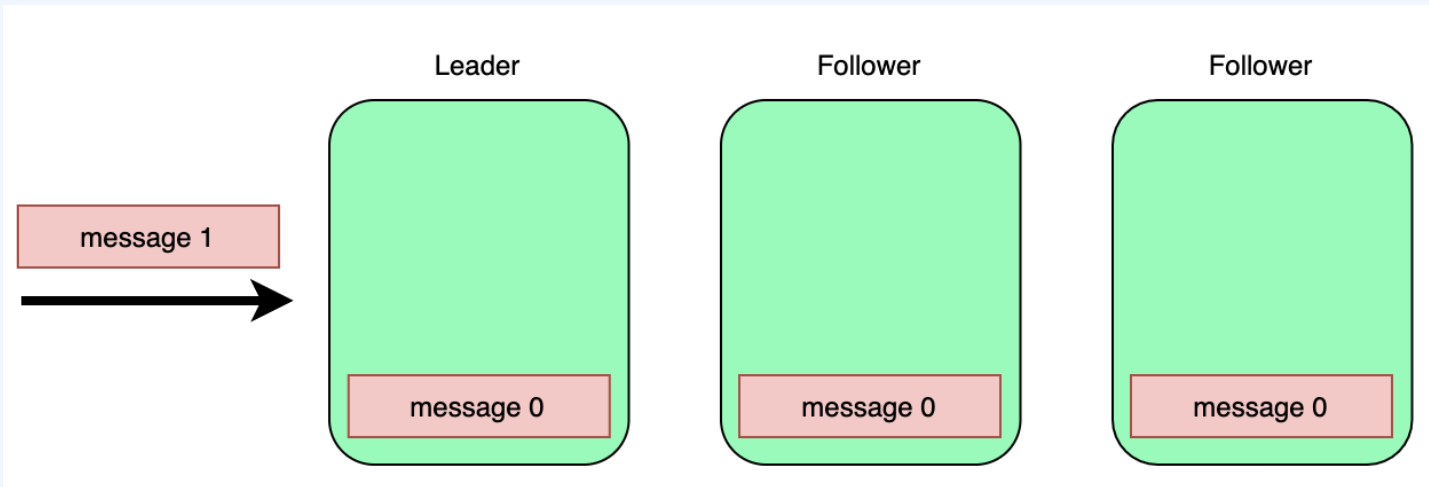
- 메시지가 들어오는 상황을 자세히 살펴보자



06. ISR에 대한 이해

ISR (In Sync Replica)

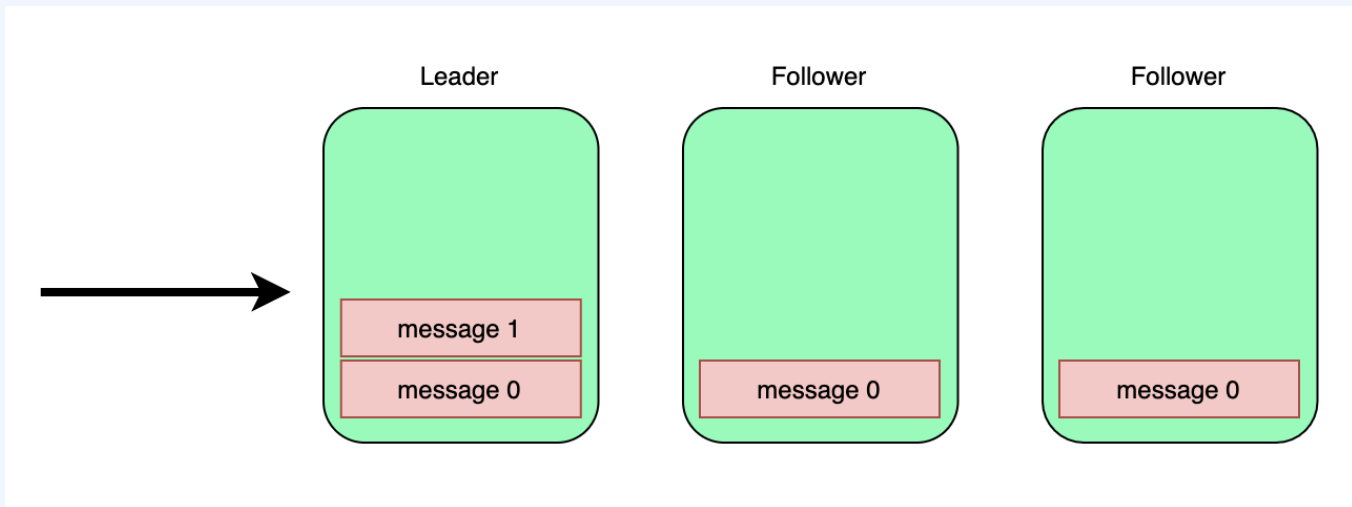
- 새 메시지가 들어오는 상황



06. ISR에 대한 이해

ISR (In Sync Replica)

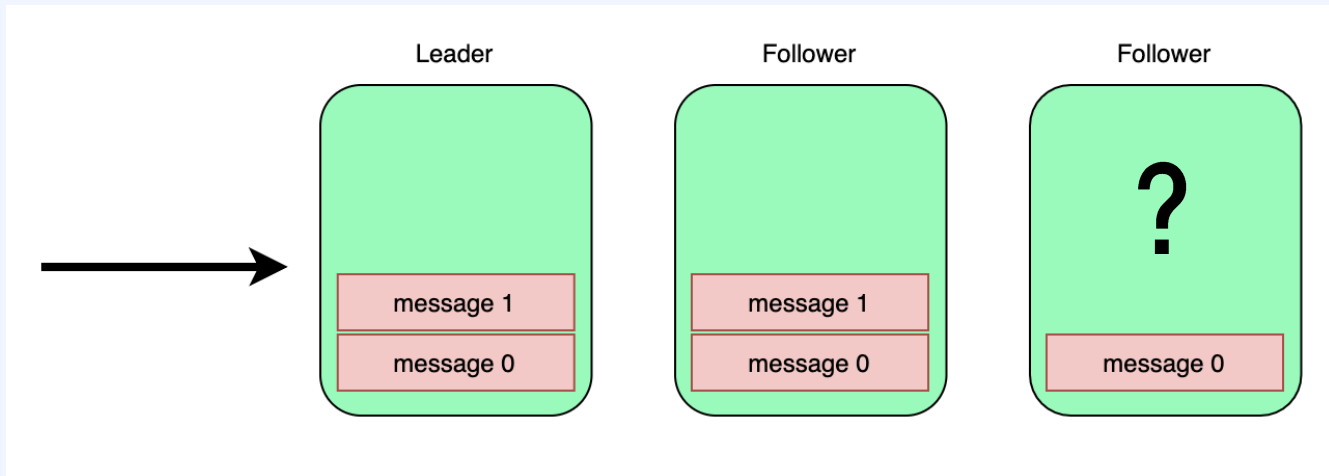
- 새 메시지가 들어오는 상황



06. ISR에 대한 이해

ISR (In Sync Replica)

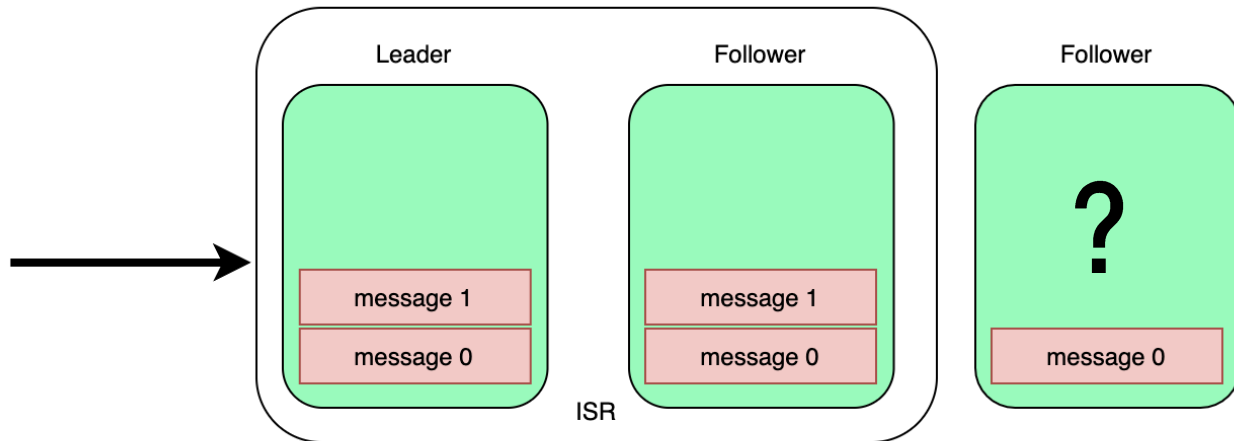
- 새 메시지가 들어오는 상황



06. ISR에 대한 이해

ISR (In Sync Replica)

- 새 메시지가 들어오는 상황



06. ISR에 대한 이해

ISR (In Sync Replica)

- ISR에 속해 있는 구성원만이 리더의 자격을 가진다
- 리더 파티션이 다운되었을때, 제 기능을 하지 못할 때,
ISR에 있는 파티션 중 하나가 리더로 승격된다
- 리더는 팔로워들이 주기적으로 데이터를 싱크하는지 확인
일정 주기 (replica.lag.time.max.ms)만큼 확인 후, 팔로워에서 요청이 없으면
리더는 해당 팔로워의 이상을 감지한다
해당 팔로워는 리더의 역할을 대신할 수 없으므로 ISR 그룹에서 해당 팔로워를 추방

Chapter 3. Kafka 프로듀서

Chapter 3. Kafka 프로듀서

01. 카프카 프로듀서의 이해

01. 카프카 프로듀서의 이해

카프카 프로듀서

- 카프카에 데이터를 저장하는 주체
- 카프카에 데이터를 전송하는 애플리케이션, 서버를 모두 프로듀서라고 부름
- 주요 기능은 1. 각각의 메시지를 토픽 파티션에 매핑
2. 파티션 리더에 write 요청을 보내는 것이다
- 키 값을 정하면 해당 키를 가진 모든 메시지를 동일한 파티션으로 전송 가능
- 만약 키 값을 입력하지 않으면, 파티션은 라운드 로빈 방식으로 파티션에 균등하게 분배

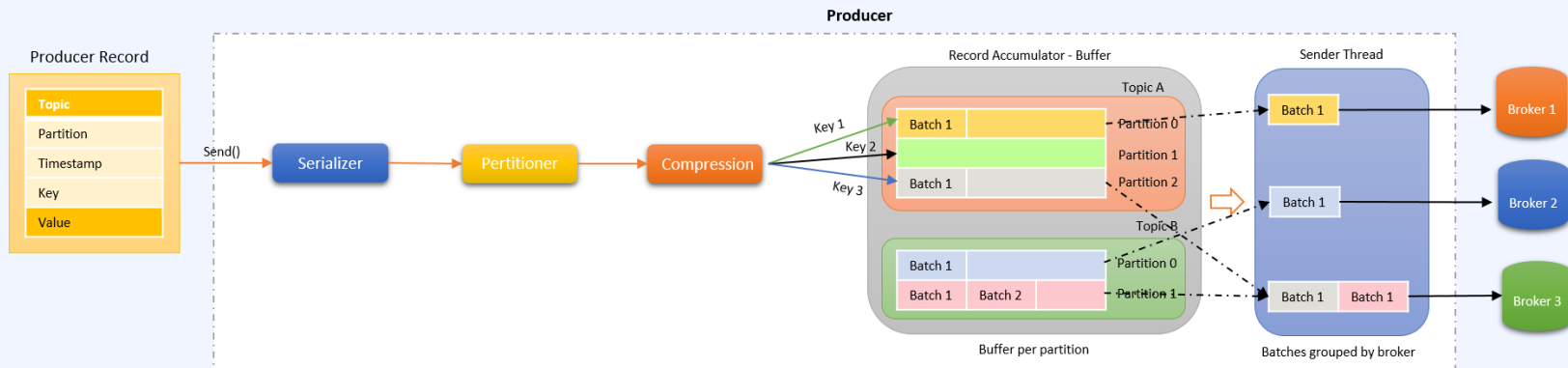
01. 카프카 프로듀서의 이해

카프카 프로듀서

- 프로듀서는 반드시 리더 파티션이 있는 브로커와 통신
- 카프카 브로커로 데이터를 전송할 때 파티셔너, 배치 생성 을 거치게 됨

01. 카프카 프로듀서의 이해

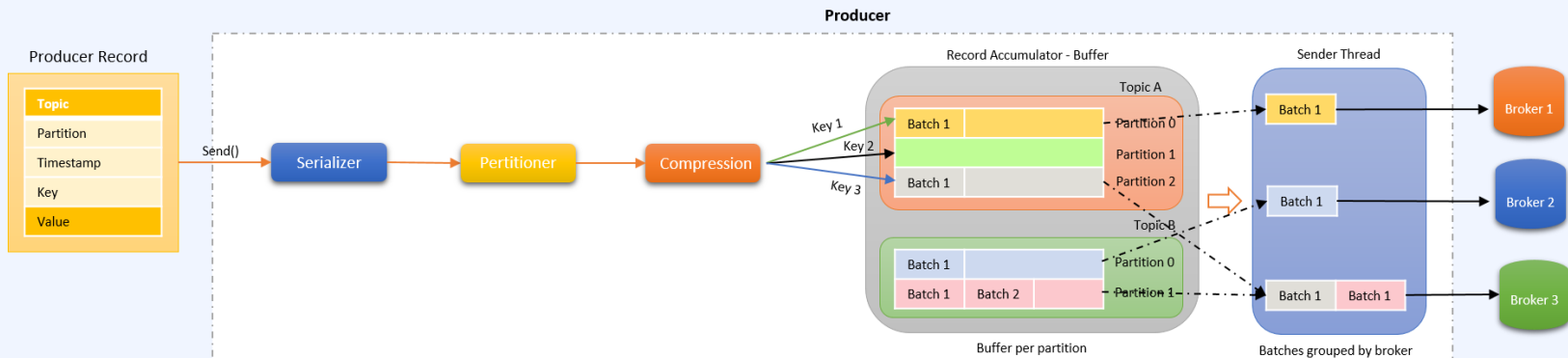
카프카 프로듀서 구조



<https://www.linkedin.com/pulse/kafka-producer-overview-sylvester-daniel/>

01. 카프카 프로듀서의 이해

카프카 프로듀서 구조

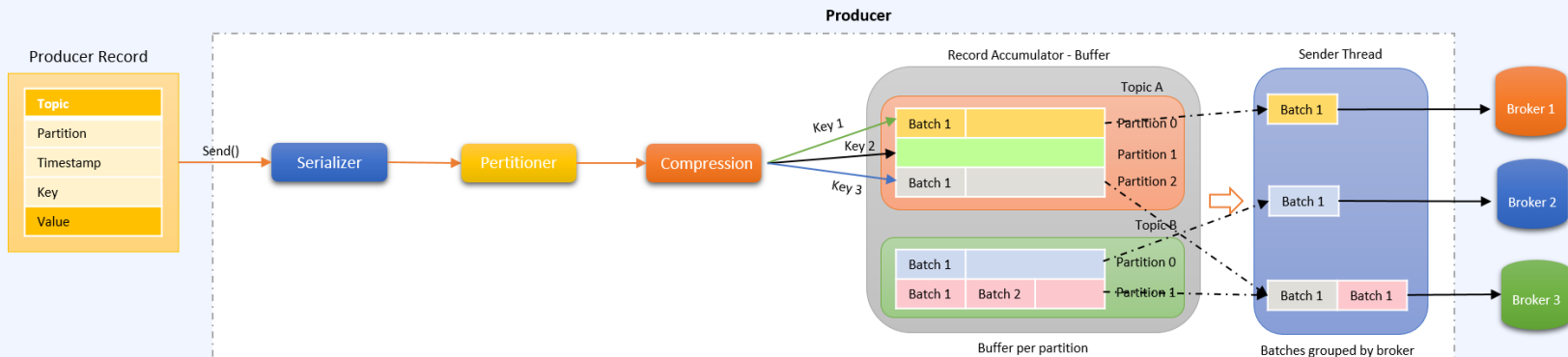


<https://www.linkedin.com/pulse/kafka-producer-overview-sylvester-daniel/>

- 오프셋은 레코드가 생성될 때에는 포함되지 않는다
- 토픽과 메세지 값만 있어도 전송하는데에는 이상이 없음

01. 카프카 프로듀서의 이해

카프카 프로듀서 구조

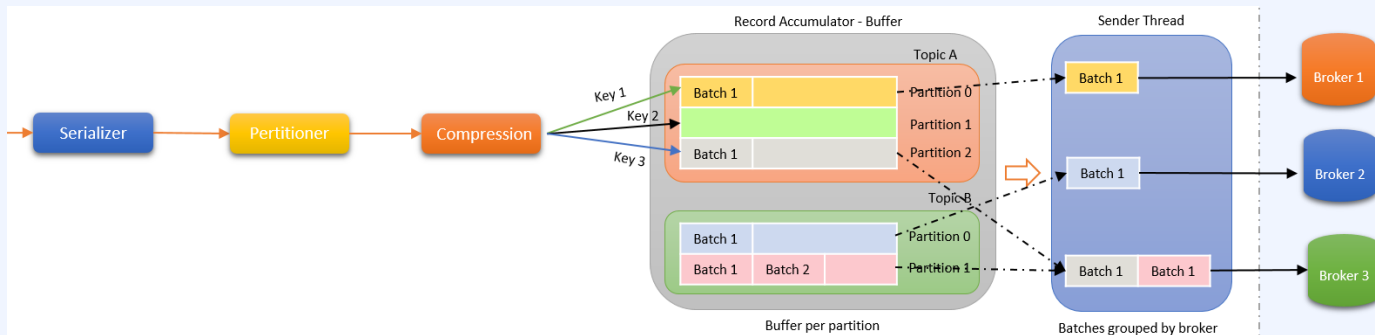


<https://www.linkedin.com/pulse/kafka-producer-overview-sylvester-daniel/>

- send()를 통해 레코드를 전송...(?)
- 실제 전송되는 건 뒤의 처리가 끝난 이후
- Partitioner, Accumulator, Sender가 끝나야 전송이 실제로 이루어짐

01. 카프카 프로듀서의 이해

카프카 프로듀서 구조

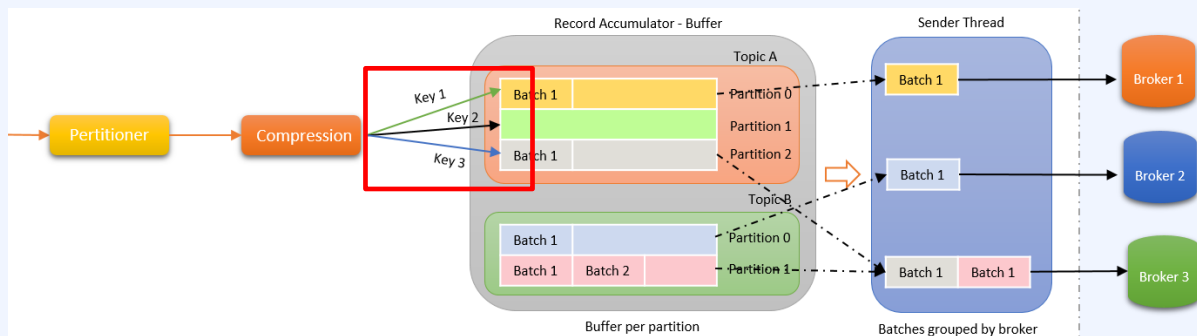


<https://www.linkedin.com/pulse/kafka-producer-overview-sylvester-daniel/>

- **Serializer** : byte array로 변환
Serializer가 지정된 설정을 통해 처리
- **Serializer**는 `key.serializer`, `value.serializer` 설정값으로 지정하거나,
프로듀서 생성 시 지정가능

01. 카프카 프로듀서의 이해

카프카 프로듀서 구조

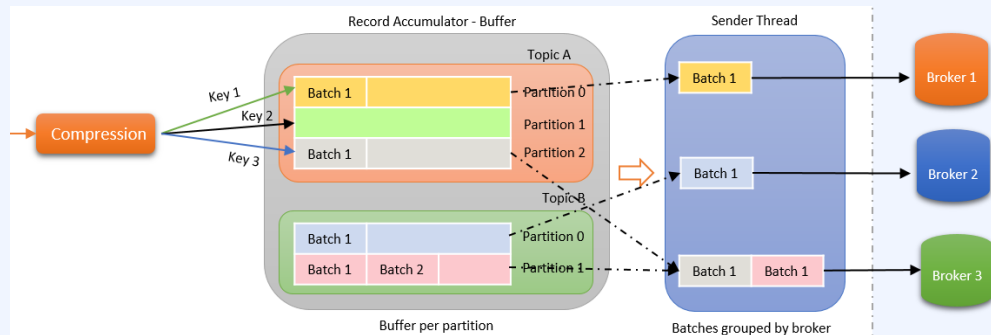


<https://www.linkedin.com/pulse/kafka-producer-overview-sylvester-daniel/>

- **Partitioner** : 어떤 파티션으로 보낼 지 설정, 기본 값은 **DefaultPartitioner**
데이터의 순서 보장 => 메시지 키가 있다면 특정 파티션으로 데이터를 전송

01. 카프카 프로듀서의 이해

카프카 프로듀서 구조

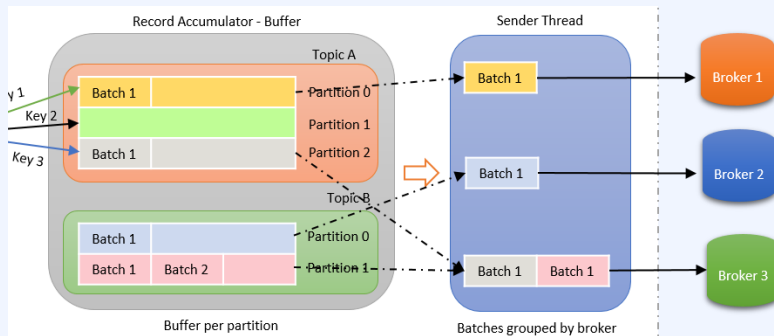


<https://www.linkedin.com/pulse/kafka-producer-overview-sylvester-daniel/>

- **Compression** : 사용자가 전송하려는 Record는 압축을 통해 네트워크 전송 비용을 줄일 수 있고 저장 비용도 줄일 수 있다
- `compression.type`을 설정하여 압축 시 사용할 코덱을 지정할 수 있다. 기본값은 none
- gzip
- snappy
- lz4

01. 카프카 프로듀서의 이해

카프카 프로듀서 구조

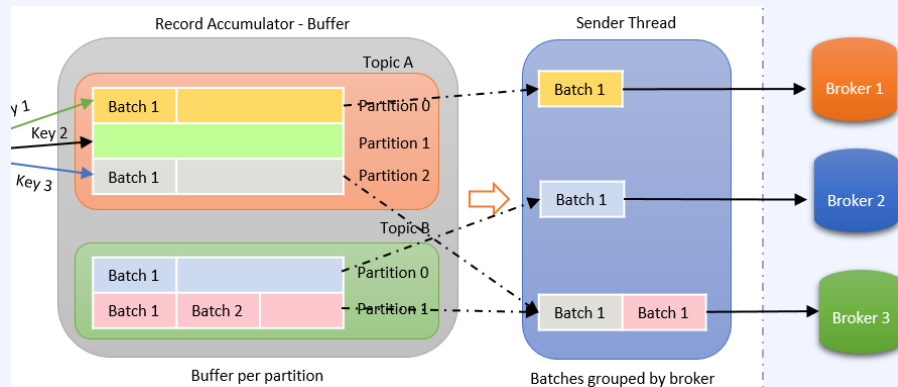


<https://www.linkedin.com/pulse/kafka-producer-overview-sylvester-daniel/>

- **Accumulator**: 네트워크 전송은 매우 무거운 처리
- 프로듀서는 지정된 만큼 메시지를 저장했다가 한 번에 브로커로 전달 => 높은 처리량 달성 가능
- Record Accumulator(RA)가 담당하여 처리
- RA는 각 토픽 파티션에 대응하는 배치 큐(Batch Queue)를 구성하고 메시지들을 레코드 배치(Record Batch) 형태로 묶어 큐에 저장

01. 카프카 프로듀서의 이해

카프카 프로듀서 구조

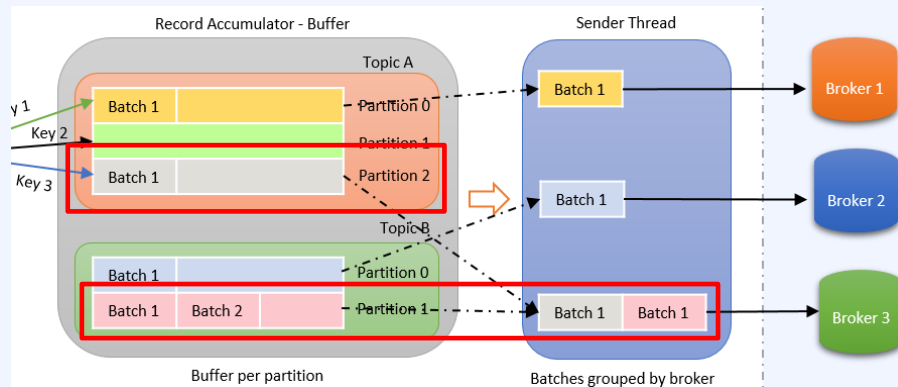


<https://www.linkedin.com/pulse/kafka-producer-overview-sylvester-daniel/>

- **Sender**: 각 배치 큐에 저장된 레코드 배치들은 때가 되면 각각 브로커에 전달됨
- 이 과정을 Sender가 처리
- Sender는 스레드 형태로 구성, 관리자가 설정한 특정 조건에 만족한 레코드 배치를 브로커로 전송

01. 카프카 프로듀서의 이해

카프카 프로듀서 구조



<https://www.linkedin.com/pulse/kafka-producer-overview-sylvester-daniel/>

- Piggyback : 네트워크 전송 비용을 줄이기 위한 방식
- 브로커에 네트워크 전송 요청을 보낸 후 Sender는 설정 값에 따라 브로커의 응답을 기다리거나 기다리지 않음
- 기다리지 않는 설정인 경우, 메시지 전송 끝
- 응답을 기다리는 경우, 메시지 전송 성공 여부를 응답으로 받는다

Chapter 3. Kafka 프로듀서

02. 카프카 프로듀서의 옵션들

02. 카프카 프로듀서의 옵션들

주요 옵션들

- bootstrap.servers

카프카 클러스터에 연결을 하기 위한 호스트와 포트 정보를 입력

[호스트 이름:포트, ...] 여러 호스트 입력을 추천

- buffer.memory

프로듀서가 Kafka서버로 데이터를 보내기 위해 대기할 수 있는 전체 메모리 바이트

- batch.size

같은 파티션으로 보내는 여러 데이터를 배치로 보내려고 시도, 정의된 크기보다 큰 데이터는 배치 전송을 시도 하지 않음.고가용성 필요하다면 배치 사이즈를 주지 않는다.

02. 카프카 프로듀서의 옵션들

주요 옵션들

- `key.serializer`

레코드의 메시지 키를 직렬화 하는 클래스 지정
이 `serializer`에 따라 역직렬화

- `value.serializer`

레코드의 메시지 값을 직렬화 하는 클래스 지정
이 `serializer`에 따라 역직렬화

02. 카프카 프로듀서의 옵션들

주요 옵션들

- `linger.ms`

Accumulator에 있는 배치를 전송하기 전까지 기다리는 최소 시간
배치를 최대한 모아서 보내고 싶다면 값을 늘려준다

- `max.request.size`

프로듀서가 보낼 수 있는 최대 메시지 바이트 사이즈
기본값은 1MB

- `retries`

브로커로부터 에러를 전달 받고 재전송을 시도하는 횟수
기본값은 2147483647로 매우 큼

02. 카프카 프로듀서의 옵션들

주요 옵션들

- `max.in.flight.requests.per.connections`
한 번에 요청하는 최대 커넥션 개수
설정 값만큼 동시 요청, 기본값 5
- `max.request.size`
프로듀서가 보낼 수 있는 최대 메세지 바이트 사이즈
기본값은 1MB
- `delivery.timeout.ms`
acks를 받지 못한 메세지에 대해 설정된 시간만큼 retry를 진행
시간이 지나도록 받지 못한다면 메세지는 fail됨

02. 카프카 프로듀서의 옵션들

주요 옵션들

- `*acks`

0, 1, -1(all) 중 하나로 선택. 기본 값은 1

프로듀서가 전송한 데이터가 브로커들에 정상적으로 들어갔는지 전송 성공 여부를 확인한다.

- `partitioner.class`

파티셔너 클래스를 지정.

기본값은 `org.apache.kafka.clients.producer.internals.DefaultPartitioner`

2.5.0이후 `UniformStickyPartitioner`

- `enable.idempotence`

멥등성 프로듀서 동작여부. 기본값 `false` 3.0.0.이후 `true`로 변경됨

- `transaction.id`

레코드를 트랜잭션 단위로 묶을지 여부 기본값은 `null`

Chapter 3. Kafka 프로듀서

03. 카프카 프로듀서 acks

03. 카프카 프로듀서 acks

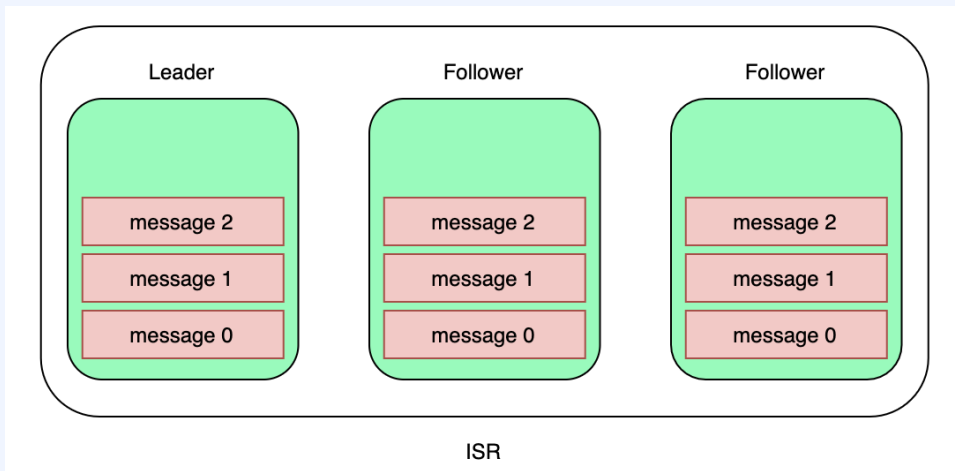
acks

- acks = 0

0, 1, -1(all) 중 하나로 선택. 기본 값은 1

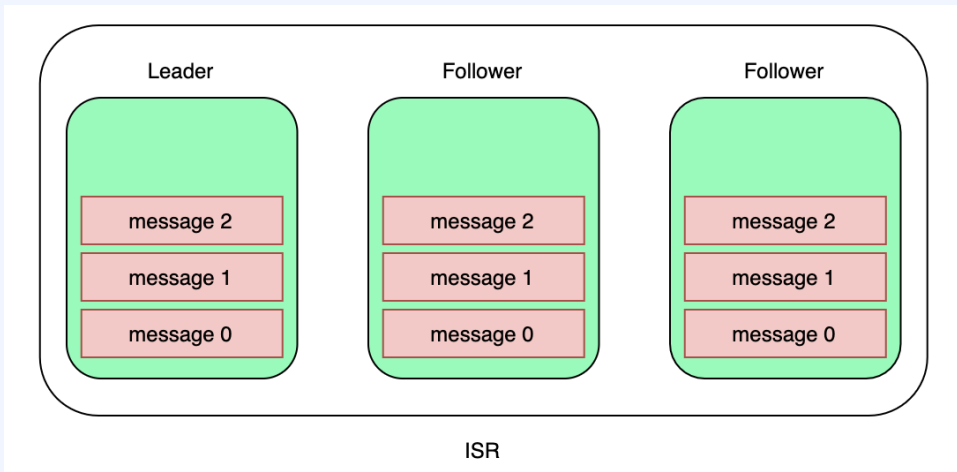
프로듀서가 전송한 데이터가 브로커들에 정상적으로 들어갔는지 전송 성공 여부를 확인한다.

- ISR(In-Sync-Replica)과 연관이 되어 있음



03. 카프카 프로듀서 acks

acks



- 리더 - 팔로워 파티션 간 복제 하는데 시간이 소요되기 때문
- 리더 파티션에 데이터가 적재된 다음 팔로워 파티션이 복제할 때 시간이 소요되기 때문에 오프셋 차이가 발생
- => Replication lag

03. 카프카 프로듀서 acks

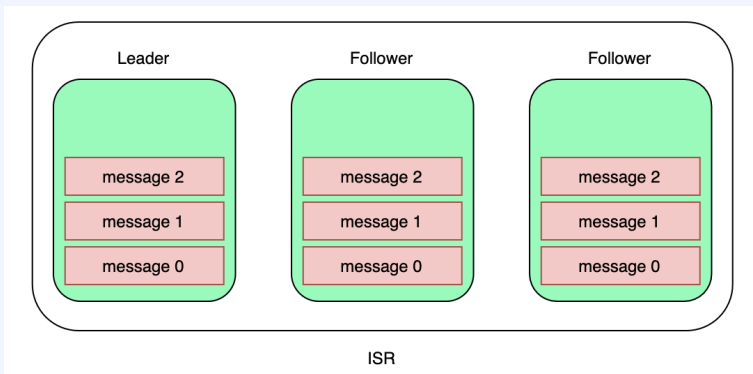
acks

- acks 옵션 0, 1, -1(all) 뒤로 갈수록 신뢰성이 높음
복제의 정도를 확인하는 것
- acks 옵션에 따라 처리 속도에도 차이가 있어 주의해야 함
- Replication factor가 1인 경우에는 변화가 거의 없음
- 보통 2 이상 설정하므로 acks 별 동작 방식의 이해가 필요

03. 카프카 프로듀서 acks

acks = 0

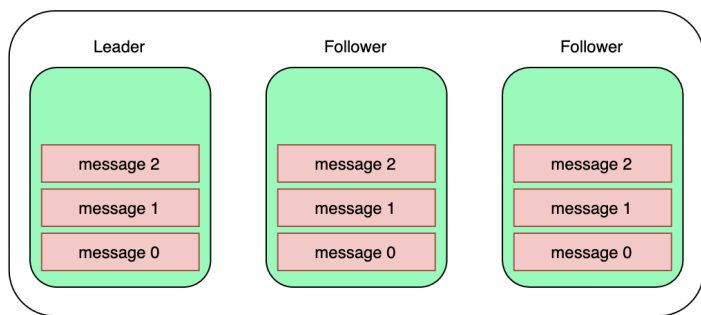
- 신뢰성 정도가 가장 낮음
즉, 프로듀서가 리더 파티션으로 데이터를 전송만 한 것
- 실제 저장까지 되었는지 여부는 확인하지 않음
- send() 호출하면 끝
- 데이터 전송 속도는 1, -1 보다 훨씬 빠름
- 데이터 유실을 감당할 수 있는 경우에 사용하면 좋음
ex) 물류 차량 위치



03. 카프카 프로듀서 acks

acks = 1

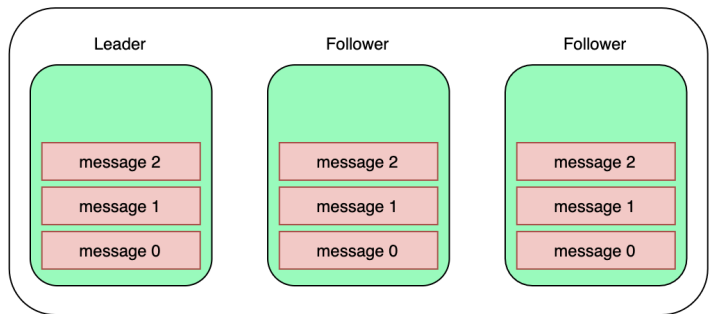
- 신뢰성 정도가 중간
- 프로듀서가 보낸 데이터가 리더 파티션에 정상적으로 적재되었는지를 확인한다
- 적재가 되지 않았다면, 될 때 까지 시도할 수 있다
- 그러나 팔로워 파티션에까지 데이터가 적재되었다는 걸 보장하지 않기 때문에 유실이 발생할 수 있다.
- 팔로워 파티션에 데이터 적재가 되기 전에 리더 파티션 브로커가 다운된다면?



03. 카프카 프로듀서 acks

acks = -1 (all)

- 신뢰성 정도가 가장 높음
- 프로듀서가 보낸 데이터가 리더, 팔로워 파티션에 정상적으로 적재되었는지를 확인한다
min.insync.replicas(토픽) 옵션값에 따라 확인함
- 처리되는 속도는 매우 느리다
꼭 필요한 경우에만 사용
- 브로커 장애에도 데이터 저장을 보장할 수 있음



Chapter 4. Kafka 컨슈머

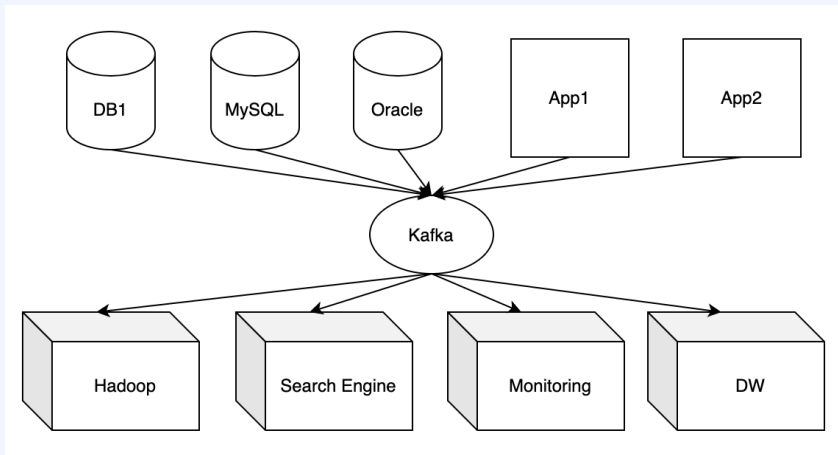
Chapter 4. Kafka 컨슈머

01. 카프카 컨슈머의 이해

01. 카프카 컨슈머의 이해

카프카 컨슈머

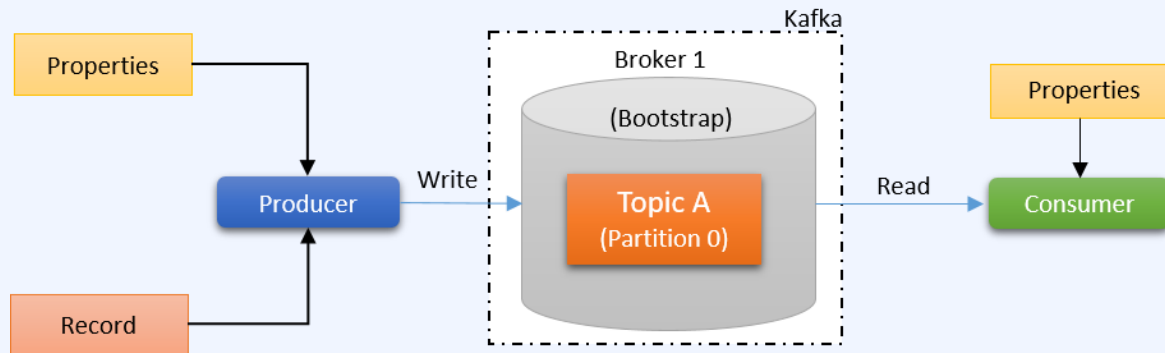
- 카프카에 토픽에 저장된 데이터를 가져오는 애플리케이션, 서버 등등
- 저장된 데이터를 가져오는 행위를 Read 읽는다고 부르기도 하고 Subscribe 구독한다고도 함
- Pub Sub 구조의 Sub에 해당



01. 카프카 컨슈머의 이해

카프카 컨슈머 특징

- Polling 구조
- Multi-app Consumption
- 컨슈머 그룹

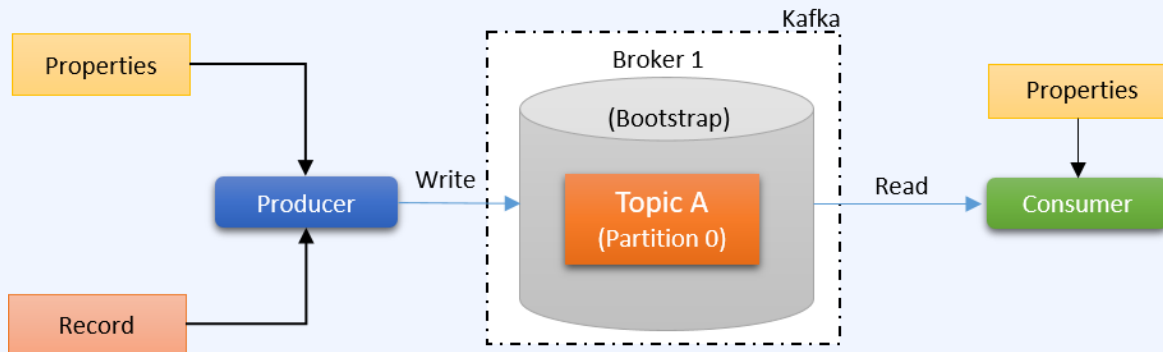


<https://www.linkedin.com/pulse/kafka-consumer-overview-sylvester-daniel/>

01. 카프카 컨슈머의 이해

Polling 구조

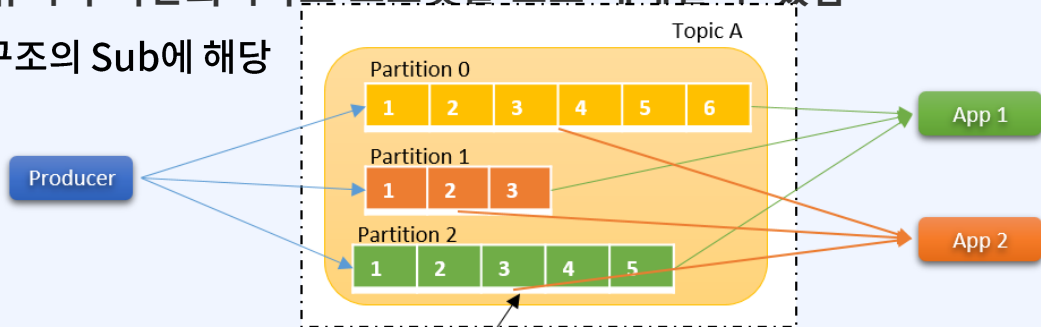
- 메세지 큐들과는 다른 점, ex) RabbitMQ
메세지 큐들은 메세지를 푸시 하는 방식
- 카프카는 Polling 방식
- 컨슈머가 원하는 만큼 카프카 브로커에 요청함



01. 카프카 컨슈머의 이해

Multi-app Consumption

- 한 토픽에 대해 서로 다른 앱이 각자의 데이터를 가져와서 읽을 수 있음
- 메시지가 삭제되지 않기 때문에 가능한 것
- 컨슈머 오프셋(`__consumer_offset`)이라는 토픽에 각 앱들이 어디까지 읽었는지를 저장함
- 갑자기 컨슈머가 다운되더라도 오프셋을 보고 **복구** 재개할 수 있음
- Pub Sub 구조의 Sub에 해당



<https://www.linkedin.com/pulse/kafka-consumer-overview-sylvester-daniel/>

Consumed offset stored in internal “`__consumer_offset`” topic by application (Consumer group)

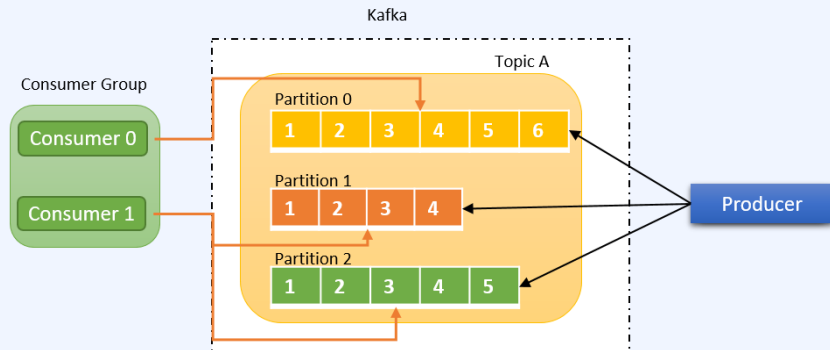
01. 카프카 컨슈머의 이해

컨슈머 그룹

- 하나 이상의 컨슈머들이 컨슈머 그룹을 이룰 수 있음
- 컨슈머 그룹단위로 묶어서 깔끔하게 처리가 가능함

한 토픽에 있는 데이터를 여러 앱 컨슈머 그룹에서 가져갈 수 있음

ex) 배송기사 데이터 -> 1. 배송기사 모바일 앱, 2. 배송기사 위치 웹 대시보드



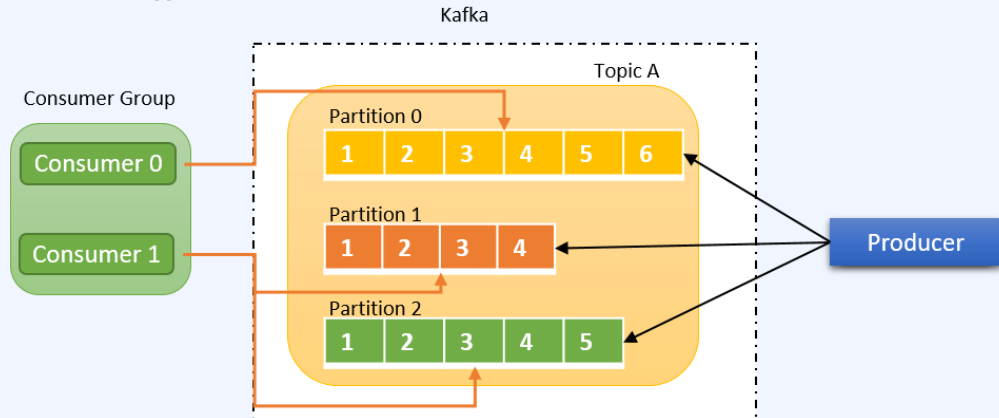
Chapter 4. Kafka 컨슈머

02. 카프카 컨슈머 그룹의 이해

02. 카프카 컨슈머 그룹의 이해

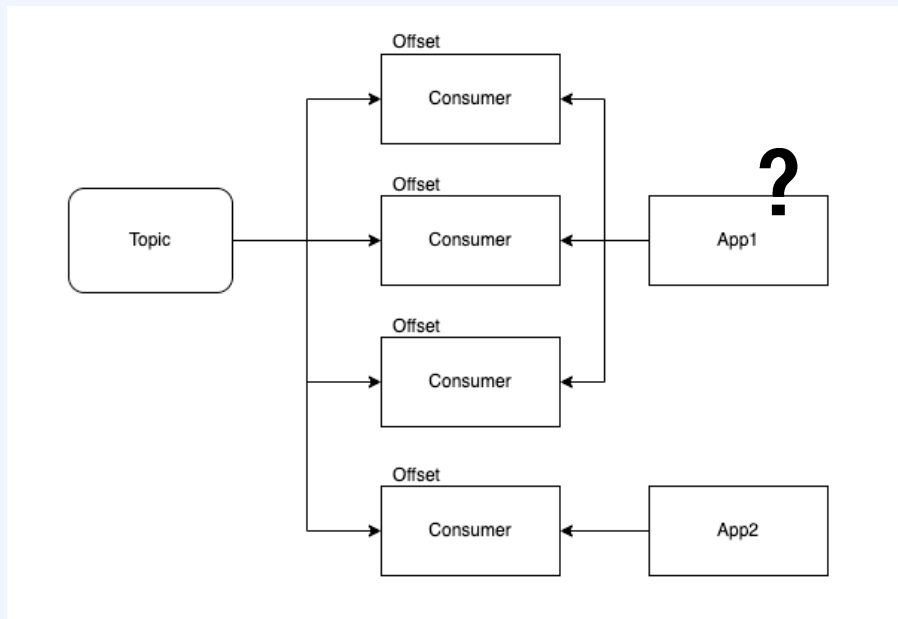
카프카 컨슈머 그룹

- 컨슈머들을 묶어서 관리하는 단위
- 컨슈머들을 컨슈머 그룹으로 격리시킨 환경환경에서 사용함으로써, 안전하게 운영할 수 있다.
- 프로듀서가 보낸 데이터를 컨슈머 그룹끼리 영향을 받지않게 처리할 수 있다.



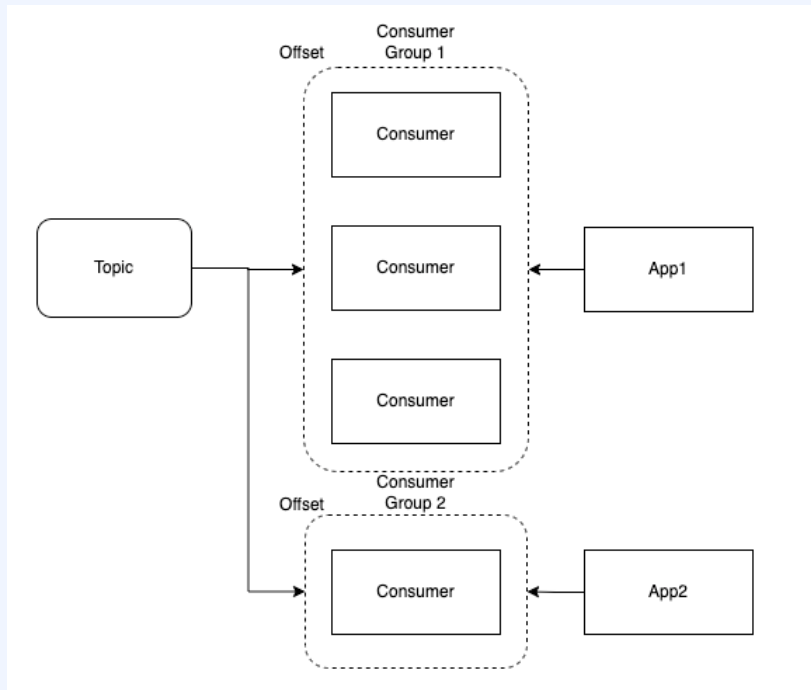
02. 카프카 컨슈머 그룹의 이해

카프카 컨슈머 그룹을 왜 사용할까
컨슈머 그룹이 없다면?!



02. 카프카 컨슈머 그룹의 이해

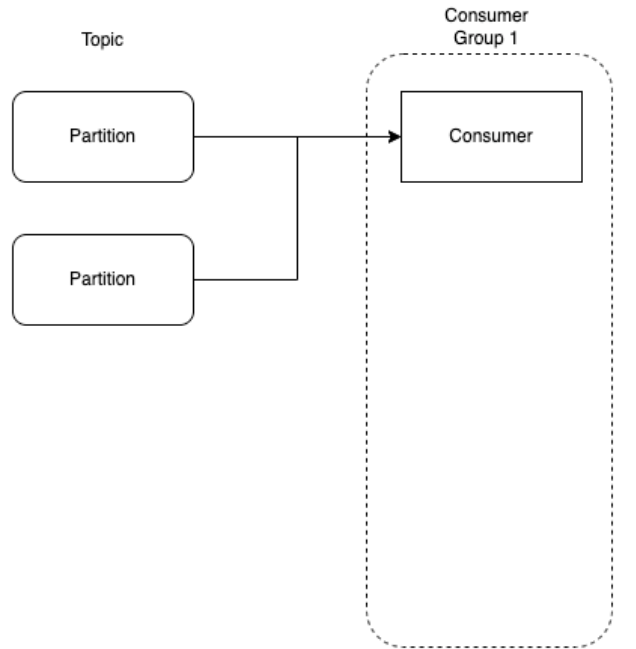
카프카 컨슈머 그룹을 왜 사용할까
컨슈머 그룹이 없다면?!



02. 카프카 컨슈머 그룹의 이해

카프카 컨슈머 그룹과 파티션

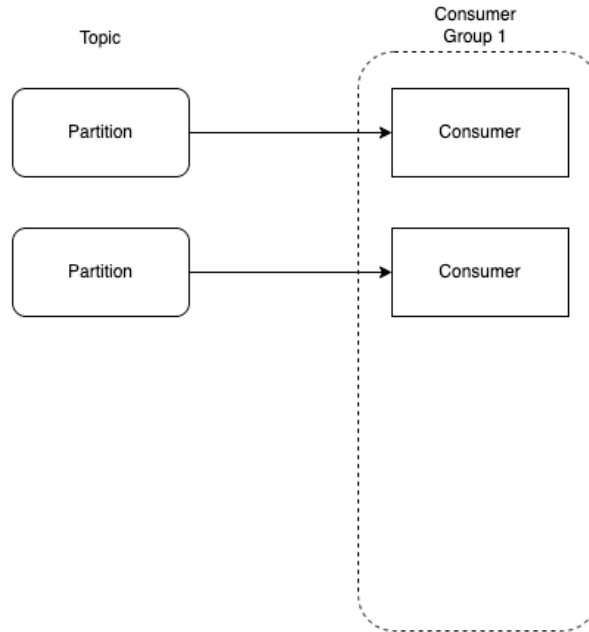
컨슈머 개수가 토픽 파티션 개수보다 적다면



02. 카프카 컨슈머 그룹의 이해

카프카 컨슈머 그룹과 파티션

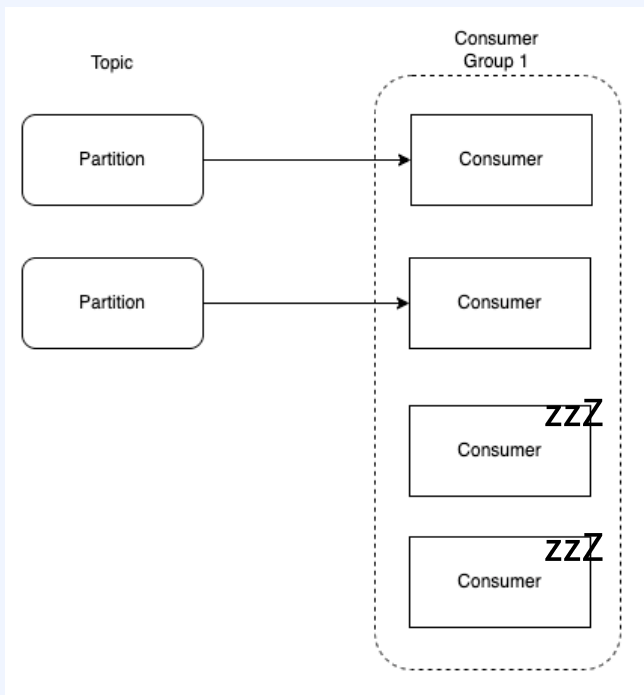
컨슈머 개수가 토픽 파티션 개수와 같다면



02. 카프카 컨슈머 그룹의 이해

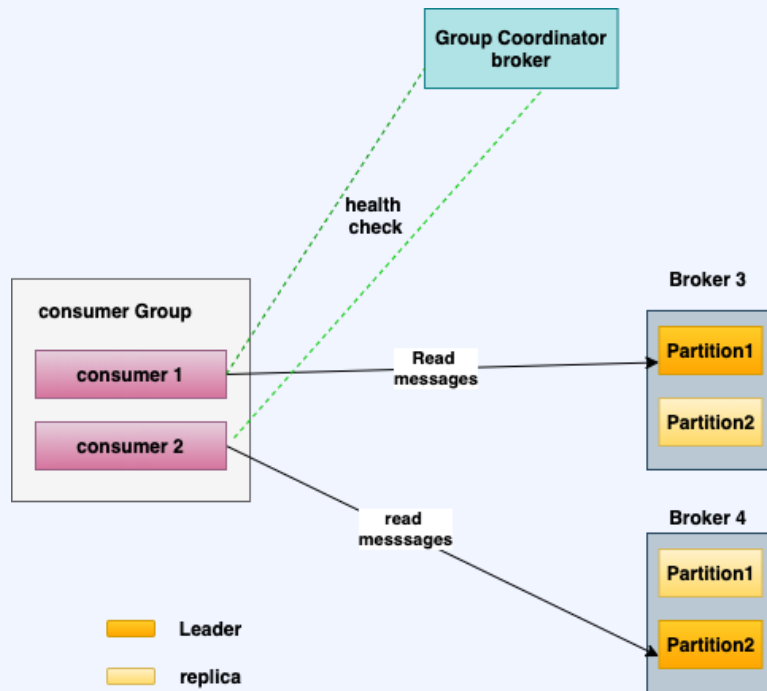
카프카 컨슈머 그룹과 파티션

컨슈머 개수가 토픽 파티션 개수보다 많다면



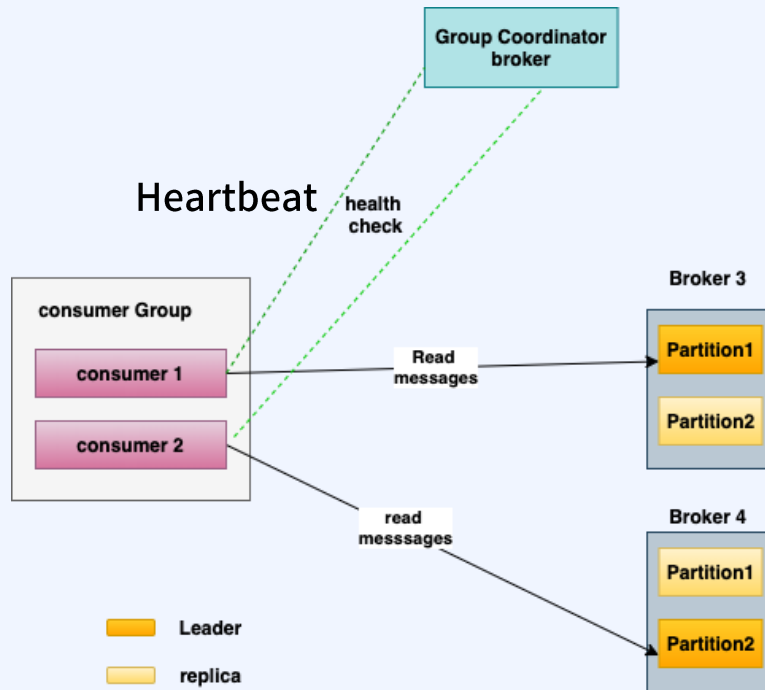
02. 카프카 컨슈머 그룹의 이해

카프카 컨슈머 그룹과 파티션 리밸런싱



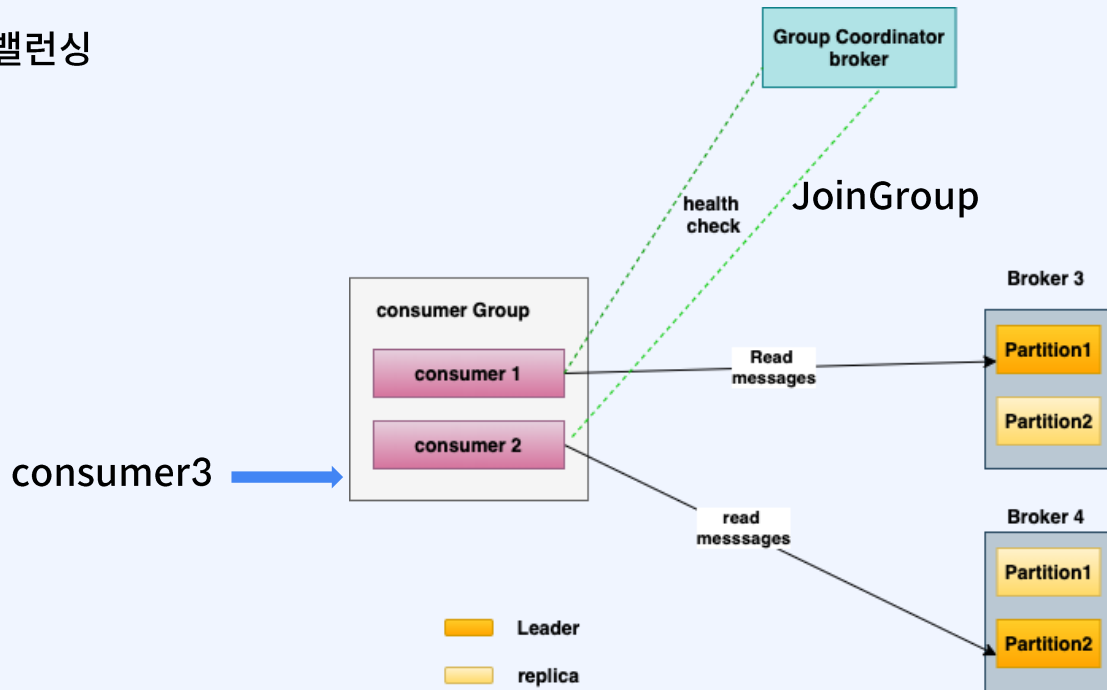
02. 카프카 컨슈머 그룹의 이해

카프카 컨슈머 그룹과 파티션 리밸런싱



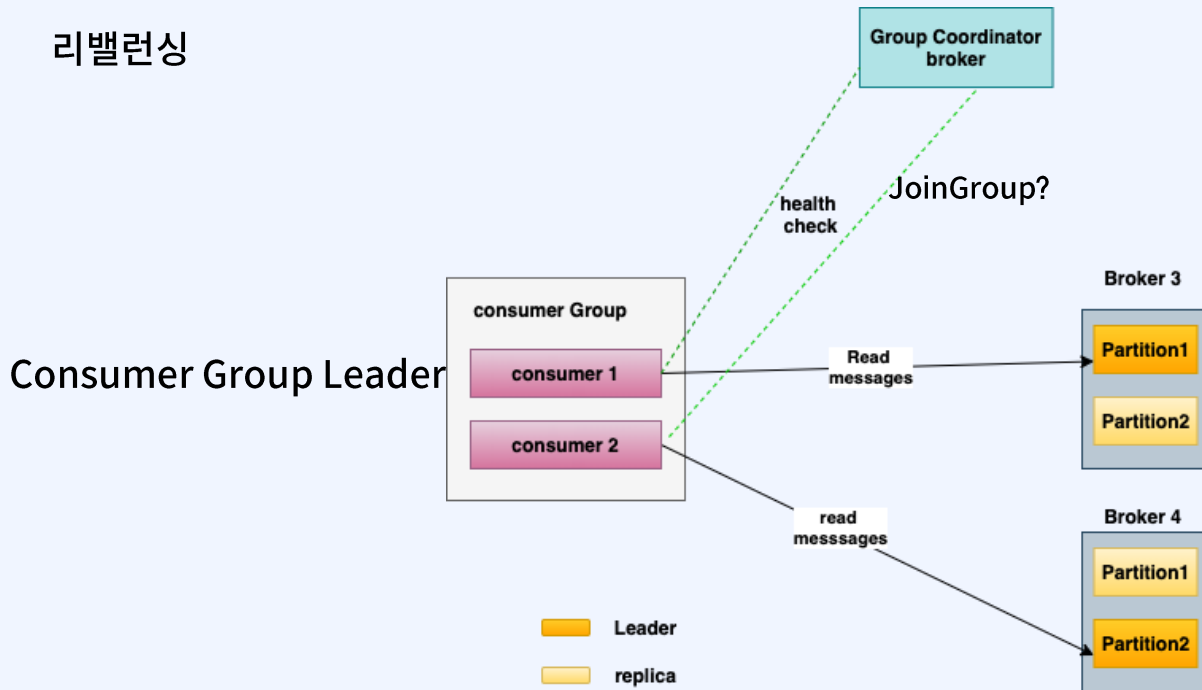
02. 카프카 컨슈머 그룹의 이해

카프카 컨슈머 그룹과 파티션 리밸런싱



02. 카프카 컨슈머 그룹의 이해

카프카 컨슈머 그룹과 파티션 리밸런싱



02. 카프카 컨슈머 그룹의 이해

카프카 컨슈머 그룹과 파티션

리밸런싱의 위험성

- 리밸런싱은 컨슈머의 소유권을 재조정하는 일
- 리밸런싱이 발생한 컨슈머 그룹 내의 모든 컨슈머의 읽기 작업은 중단 됨
- 컨슈머 측의 일시적인 서비스 중단이 발생할 수 있습니다.
실제로 파티션 개수 조정하면서 자주 발생하는 일(토픽 파티션 조정 -> 컨슈머 리밸런싱)
- 리밸런싱은 컨슈머의 멤버십의 변화는 물론 파티션 수가 증가할 때도 발생
- 컨슈머, 파티션의 추가가 필요하다면 충분한 고려 후 추가

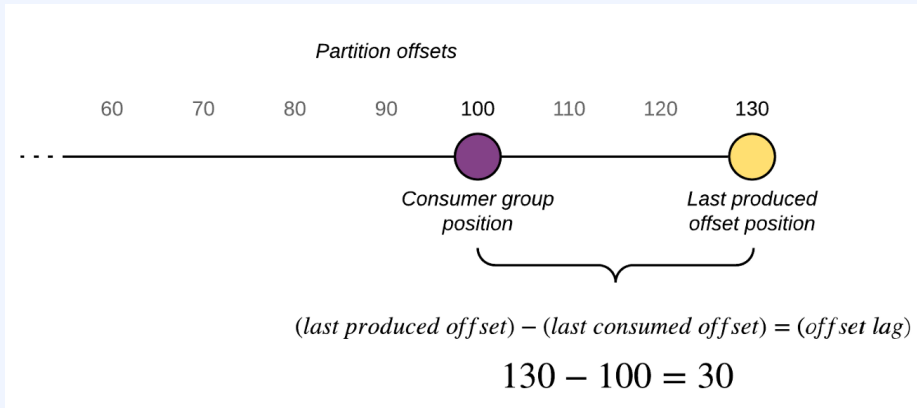
Chapter 4. Kafka 컨슈머

03. 카프카 컨슈머 랙

03. 카프카 컨슈머 랙

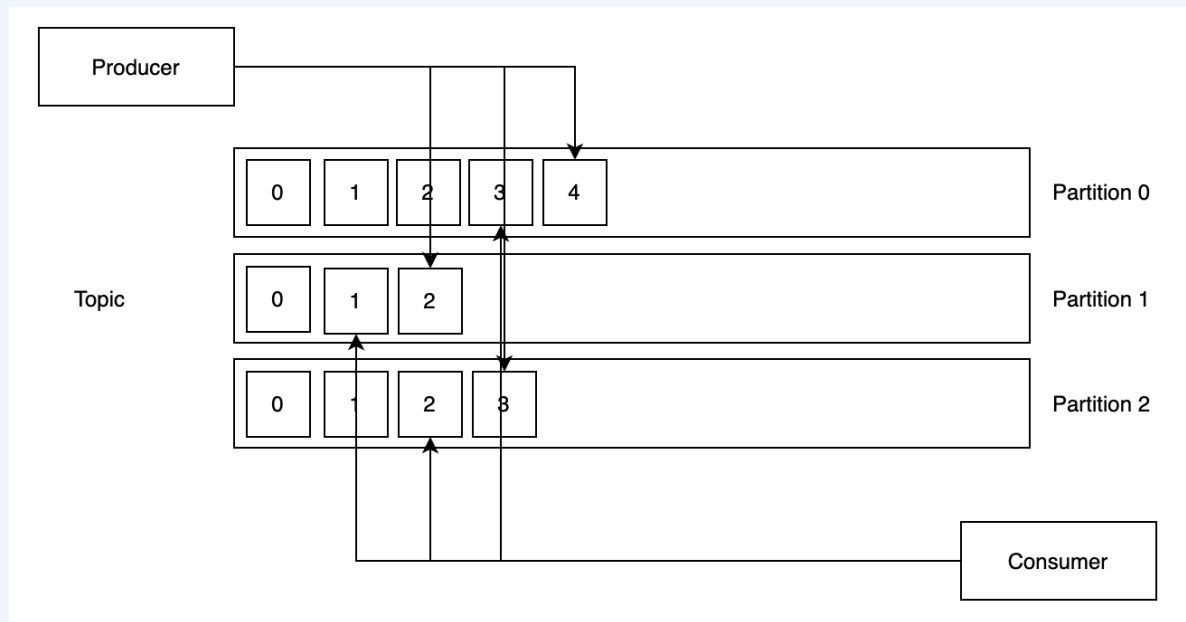
컨슈머 랙

- 컨슈머 랙이란 토픽의 최신 오프셋과 컨슈머 오프셋 간의 차이를 말함
LOG-END-OFFSET, CURRENT-OFFSET
- 프로듀서는 파티션에 데이터 저장, 컨슈머는 데이터를 가져가는 구조
- 컨슈머 랙을 통해 카프카 컨슈머의 상태를 확인할 수 있음



03. 카프카 컨슈머 락

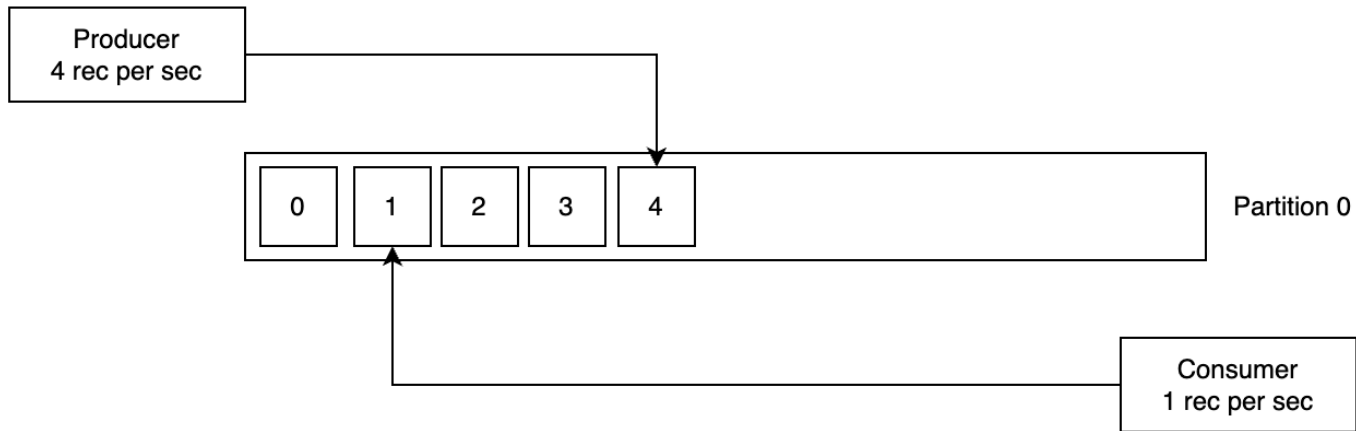
컨슈머 락



03. 카프카 컨슈머 랙

컨슈머 랙

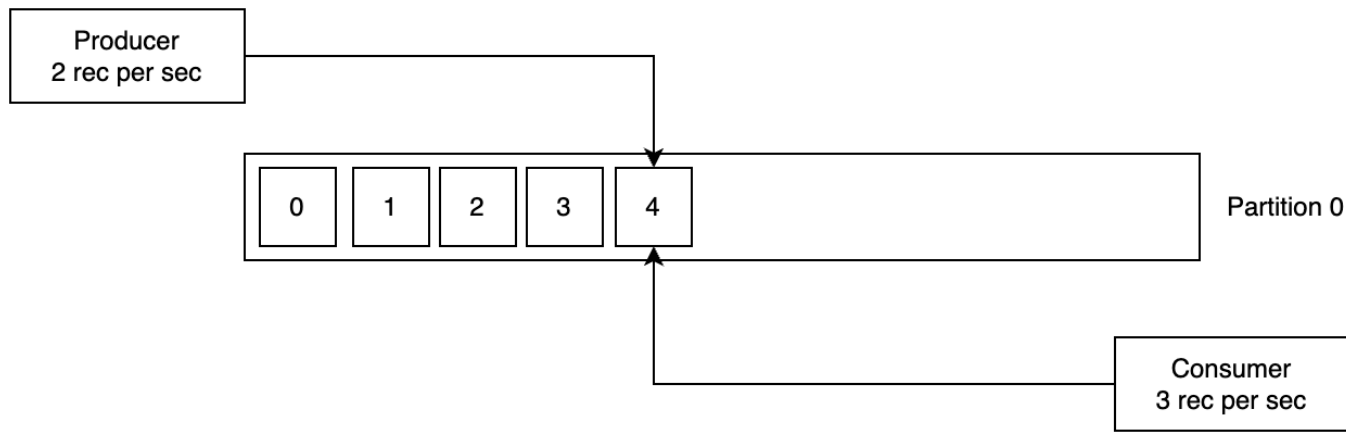
프로듀서 전송량이 컨슈머 처리량보다 크다면?



03. 카프카 컨슈머 랙

컨슈머 랙

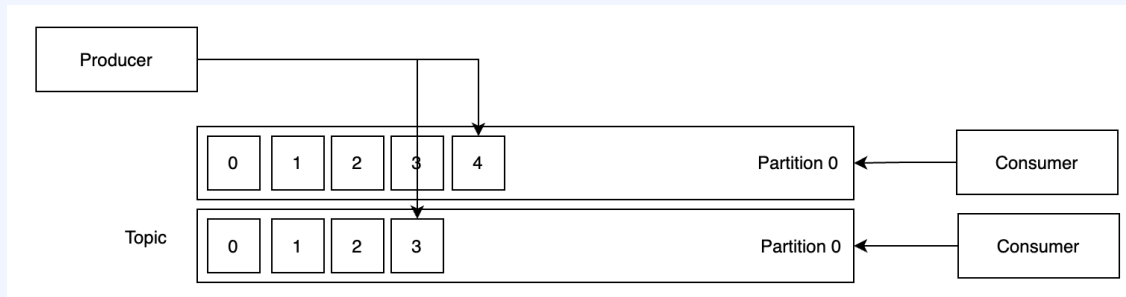
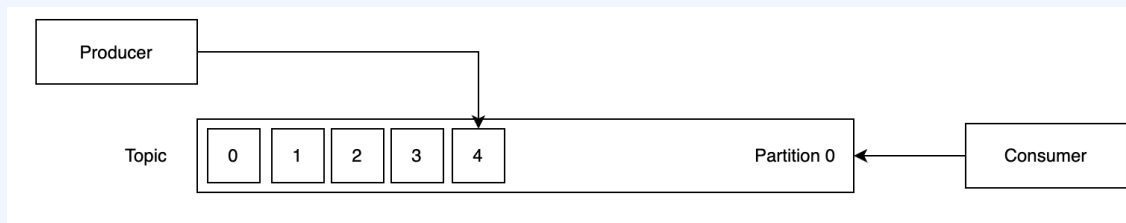
프로듀서 전송량이 컨슈머 처리량보다 작다면?



03. 카프카 컨슈머 락

컨슈머 락

컨슈머 락 해결 방법



Chapter 4. Kafka 컨슈머

04. 카프카 컨슈머 옵션들

04. 카프카 컨슈머 옵션들

컨슈머 옵션

- `bootstrap.servers` : 프로듀서와 동일하게 브로커의 정보
- `fetch.min.bytes`: 한 번에 가져올 수 있는 최소 데이터 크기
만약 지정한 크기 보다 작은 경우 요청에 응답하지 않고 데이터가 누적될 때까지 기다림
- `group.id` : 컨슈머가 속한 컨슈머 그룹을 식별하는 식별자
동일한 그룹 내의 컨슈머 정보는 모두 공유됨

04. 카프카 컨슈머 옵션들

컨슈머 옵션

- `heartbeat.interval.ms` : 하트비트가 있다는 것은 컨슈머의 상태는 active
`session.timeout.ms`와 밀접한 관계가 있음
`session.timeout.ms`보다 낮은 값으로 설정해야 함(일반적으로 `session.timeout.ms`의 1/3로 설정)
- `session.timeout.ms` : 이 시간을 이용해, 컨슈머가 종료된 것인지를 판단
컨슈머는 주기적으로 하트비트를 전송하는데, 만약 이 시간 전까지 하트비트를 보내지 않았다면,
해당 컨슈머는 종료된 것으로 간주하고 컨슈머 그룹에서 제외. 리밸런싱을 시작
- `max.partition.fetch.bytes` : 파티션당 가져올 수 있는 최대 크기
- `enable.auto.commit` : 백그라운드로 주기적으로 오프셋을 커밋

04. 카프카 컨슈머 옵션들

컨슈머 옵션

- `auto.offset.reset`: 카프카에서 초기 오프셋이 없는 경우, 또는 현재 오프셋이 더 이상 존재하지 않는 경우에 아래 옵션으로 reset
 - `earliest`: 가장 초기의 오프셋값으로 설정
 - `latest`: 가장 마지막의 오프셋값으로 설정
 - `none`: 이전 오프셋값을 찾지 못하면 에러
- `fetch.max.bytes`: 한 번의 fetch 요청으로 가져올 수 있는 최대 크기
- `group.instance.id`: 컨슈머의 고유한 식별자. 설정 시, static 멤버로 간주되어 불필요한 리밸런싱을 하지 않음

04. 카프카 컨슈머 옵션들

컨슈머 옵션

- `max.poll.records` :한 번의 `poll()`요청으로 가져오는 최대 메시지 수
- `fetch.max.watt.ms` :`fetch.mln.bytes`에 의해 설정된 데이터보다 적은 경우 요청에 대한 응답을 기다리는 최대 시간